

05_tarea_grafica_radial_pokemon

April 27, 2026

1 Tarea paso a paso: Gráfico Radial (Radar Chart)

¡ATENCIÓN! Lee esto antes de empezar: Esta tarea está diseñada paso a paso para que afiances los conceptos vistos en clase sobre los **Radar Charts**. Su objetivo es que practiques cómo comparar visualmente múltiples variables numéricas entre distintas categorías de manera clara y efectiva.

Nota importante sobre la Inteligencia Artificial: Es tentador copiar y pegar el enunciado de estos ejercicios en ChatGPT o Claude para que te den la solución... **Hacer eso perjudicará gravemente tu aprendizaje. (CODEA UN POCO QUE TE ENTERES QUE HACES).** La única forma de asimilar la sintaxis de las herramientas de visualización es escribiéndolas. **Usa tu cerebro, actúa paso a paso, equivócate, lee la documentación de Plotly, busca tu fallo y ¡aprende de él!**

1.1 Preparación y Datos

Para esta tarea vamos a utilizar el famoso dataset “**Pokémon**”, un conjunto de datos que recoge las estadísticas de combate de todos los Pokémon de las primeras 6 generaciones. Es un dataset ideal para un **Radar Chart** porque cuenta con 6 variables numéricas de combate (HP, Attack, Defense, Sp. Atk, Sp. Def, Speed) y una columna categórica (Type 1) que nos permite comparar tipos entre sí.

1. Descarga el dataset de Kaggle: [Pokémon - Kaggle](#)
2. Guarda el archivo `Pokemon.csv` en la carpeta `data/` de tu proyecto.
3. En la siguiente celda, importa las librerías necesarias (`pandas`, `matplotlib.pyplot`, `plotly.graph_objects`) y carga el CSV en un DataFrame llamado `df`.

```
[1]: import pandas as pd
import matplotlib.pyplot as plt
import plotly.graph_objects as go
from math import pi

# CARGA DE DATOS
df = pd.read_csv("data/Pokemon.csv")

df.head(5)
```

```
[1]:      #      Name Type 1  Type 2  Total  HP  Attack  Defense  \
0  1      Bulbasaur  Grass  Poison    318  45     49     49
```

1	2	Ivysaur	Grass	Poison	405	60	62	63
2	3	Venusaur	Grass	Poison	525	80	82	83
3	3	VenusaurMega Venusaur	Grass	Poison	625	80	100	123
4	4	Charmander	Fire	NaN	309	39	52	43

	Sp. Atk	Sp. Def	Speed	Generation	Legendary
0	65	65	45	1	False
1	80	80	60	1	False
2	100	100	80	1	False
3	122	120	80	1	False
4	60	50	65	1	False

```
[2]: # Ver la información del dataset
df.describe(include="all")
```

```
[2]:
```

	#	Name	Type 1	Type 2	Total	HP \
count	800.000000	800	800	414	800.000000	800.000000
unique	NaN	800	18	18	NaN	NaN
top	NaN	Bulbasaur	Water	Flying	NaN	NaN
freq	NaN	1	112	97	NaN	NaN
mean	362.813750	NaN	NaN	NaN	435.10250	69.258750
std	208.343798	NaN	NaN	NaN	119.96304	25.534669
min	1.000000	NaN	NaN	NaN	180.00000	1.000000
25%	184.750000	NaN	NaN	NaN	330.00000	50.000000
50%	364.500000	NaN	NaN	NaN	450.00000	65.000000
75%	539.250000	NaN	NaN	NaN	515.00000	80.000000
max	721.000000	NaN	NaN	NaN	780.00000	255.000000

	Attack	Defense	Sp. Atk	Sp. Def	Speed \
count	800.000000	800.000000	800.000000	800.000000	800.000000
unique	NaN	NaN	NaN	NaN	NaN
top	NaN	NaN	NaN	NaN	NaN
freq	NaN	NaN	NaN	NaN	NaN
mean	79.001250	73.842500	72.820000	71.902500	68.277500
std	32.457366	31.183501	32.722294	27.828916	29.060474
min	5.000000	5.000000	10.000000	20.000000	5.000000
25%	55.000000	50.000000	49.750000	50.000000	45.000000
50%	75.000000	70.000000	65.000000	70.000000	65.000000
75%	100.000000	90.000000	95.000000	90.000000	90.000000
max	190.000000	230.000000	194.000000	230.000000	180.000000

	Generation	Legendary
count	800.00000	800
unique	NaN	2
top	NaN	False
freq	NaN	735
mean	3.32375	NaN

std	1.66129	NaN
min	1.00000	NaN
25%	2.00000	NaN
50%	3.00000	NaN
75%	5.00000	NaN
max	6.00000	NaN

```
[3]: # Limpieza de nulos
df = df.dropna()
df
```

```
[3]:
```

	#	Name	Type 1	Type 2	Total	HP	Attack	Defense	\
0	1	Bulbasaur	Grass	Poison	318	45	49	49	
1	2	Ivysaur	Grass	Poison	405	60	62	63	
2	3	Venusaur	Grass	Poison	525	80	82	83	
3	3	VenusaurMega Venusaur	Grass	Poison	625	80	100	123	
6	6	Charizard	Fire	Flying	534	78	84	78	
..	...	...	...	...	...	...	...	...	
795	719	Diancie	Rock	Fairy	600	50	100	150	
796	719	DiancieMega Diancie	Rock	Fairy	700	50	160	110	
797	720	HoopahHoop Confined	Psychic	Ghost	600	80	110	60	
798	720	HoopahHoop Unbound	Psychic	Dark	680	80	160	60	
799	721	Volcanion	Fire	Water	600	80	110	120	

	Sp. Atk	Sp. Def	Speed	Generation	Legendary
0	65	65	45	1	False
1	80	80	60	1	False
2	100	100	80	1	False
3	122	120	80	1	False
6	109	85	100	1	False
..	...	...	...	...	...
795	100	150	50	6	True
796	160	110	110	6	True
797	150	130	70	6	True
798	170	130	80	6	True
799	130	90	70	6	True

[414 rows x 13 columns]

1.2 Paso 1: Filtrar y Agrupar los Datos

Objetivo: Preparar el DataFrame para el gráfico radial. Necesitamos resumir las estadísticas de combate para cada tipo de Pokémon calculando la **media** de cada variable numérica.

Instrucciones: 1. Crea una lista llamada **stats** con los nombres de las 6 columnas numéricas de combate: 'HP', 'Attack', 'Defense', 'Sp. Atk', 'Sp. Def' y 'Speed'. 2. Usando **df**, selecciona únicamente las columnas de **stats** más la columna 'Type 1', agrupa por 'Type 1' y calcula la media (**.mean()**). Guarda el resultado en una variable llamada **agrupado**. 3. Muestra **agrupado**

para ver el resultado.

```
[4]: # Las 6 stats de combate
stats = ['HP', 'Attack', 'Defense', 'Sp. Atk', 'Sp. Def', 'Speed']

# Por Type 1, media de cada stat (uso media porque quiero el perfil "tipico"
# del tipo)
agrupado = df[stats + ['Type 1']].groupby('Type 1').mean()
agrupado
```

```
[4]:
```

	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed
Type 1						
Bug	58.134615	77.711538	75.730769	58.634615	71.711538	66.173077
Dark	69.809524	88.190476	74.523810	71.476190	73.238095	73.047619
Dragon	94.571429	122.571429	96.476190	116.238095	97.857143	94.666667
Electric	59.470588	68.882353	83.764706	108.882353	85.352941	84.470588
Fairy	70.000000	45.000000	90.000000	100.000000	110.000000	60.000000
Fighting	66.142857	95.857143	75.142857	82.571429	69.857143	88.285714
Fire	78.291667	92.166667	78.708333	105.458333	80.166667	76.791667
Flying	62.500000	50.000000	57.500000	71.000000	60.000000	89.000000
Ghost	71.318182	72.545455	78.090909	82.045455	70.590909	69.500000
Grass	69.837838	77.135135	71.135135	81.243243	75.351351	62.108108
Ground	80.000000	101.315789	86.263158	62.421053	67.578947	68.842105
Ice	76.818182	69.090909	61.363636	75.000000	69.090909	67.727273
Normal	70.972973	72.972973	57.540541	56.432432	57.756757	75.000000
Poison	68.692308	77.461538	70.615385	63.923077	67.307692	74.076923
Psychic	66.789474	84.473684	68.578947	98.105263	94.052632	86.736842
Rock	64.914286	90.171429	99.142857	69.200000	79.885714	61.828571
Steel	65.954545	93.318182	125.090909	68.318182	78.045455	55.545455
Water	77.867925	76.735849	77.113208	76.886792	76.622642	63.830189

1.3 Paso 2: Radar Chart con Datos Brutos

Objetivo: Construir tu primer gráfico radial con Plotly usando los datos sin normalizar, y observar qué problema aparece.

Instrucciones: 1. Selecciona 3 tipos de Pokémon para comparar. Por ejemplo: 'Fire', 'Water' y 'Grass' (el trío clásico). Guárdalos en una lista `tipos_seleccionados`. 2. Extrae las estadísticas de cada tipo como una lista de Python usando `list(agrupado.loc['Fire', :])`, y haz lo mismo para los otros dos tipos. 3. Crea un `go.Figure()` y añade un `go.Scatterpolar` por cada tipo con los parámetros `r` (los valores), `theta` (la lista `stats`), `fill='toself'` y `name` (el nombre del tipo). 4. Actualiza el layout con `fig.update_layout(...)` para añadir un título y mostrar la leyenda. 5. Muestra el gráfico con `fig.show()`. ¿Qué problema observas con las escalas?

```
[5]: # Trío clásico Fire/Water/Grass (los iniciales del juego, tienen perfiles muy
# distintos)
tipos_seleccionados = ['Fire', 'Water', 'Grass']
```

```

# Listas de cada tipo (igual que adelie/chinstrap/gentoo en la teoría)
fire = list(agrupado.loc['Fire', :])
water = list(agrupado.loc['Water', :])
grass = list(agrupado.loc['Grass', :])
print('fire ', fire)
print('water ', water)
print('grass ', grass)

# Radar Chart con Plotly
# https://plotly.com/python/radar-chart/
fig = go.Figure()
fig.add_trace(go.Scatterpolar(
    r=fire,
    theta=stats,
    fill='toself',
    name='Fire'
))

fig.add_trace(go.Scatterpolar(
    r=water,
    theta=stats,
    fill='toself',
    name='Water'
))

fig.add_trace(go.Scatterpolar(
    r=grass,
    theta=stats,
    fill='toself',
    name='Grass'
))

# Aún sin tocar el eje radial: quiero ver que arranca en ~50 en vez de en 0
fig.update_layout(
    title='Fire vs Water vs Grass (datos brutos)',
    showlegend=True
)

fig.show()

```

```

fire [78.29166666666667, 92.16666666666667, 78.70833333333333,
105.45833333333333, 80.16666666666667, 76.79166666666667]
water [77.86792452830188, 76.73584905660377, 77.11320754716981,
76.88679245283019, 76.62264150943396, 63.83018867924528]
grass [69.83783783783784, 77.13513513513513, 71.13513513513513,
81.24324324324324, 75.35135135135135, 62.108108108108105]

```

Fire vs Water vs Grass (datos brutos)



1.3.1 Interpretación del Gráfico (Datos Brutos)

Como podemos observar, aunque el gráfico ya tiene forma de radar, **el eje radial empieza en un valor muy alto** (alrededor de 50-60 en lugar de 0). Esto hace que las diferencias entre tipos parezcan más pequeñas de lo que realmente son. Además, si las variables tuviesen escalas muy distintas (por ejemplo, una en decenas y otra en miles), el gráfico sería directamente inútil.

Solución: normalizar los datos para que todos los valores estén entre 0 y 1.

1.4 Paso 3: Normalización de los Datos

Objetivo: Escalar todas las estadísticas al rango $[0, 1]$ para que el gráfico radial sea comparativo y justo.

Instrucciones: 1. Selecciona del DataFrame `df` solo las columnas de `stats` y guárdalas en `df2`. 2. Aplica la fórmula de normalización Min-Max: $(df2 - df2.min()) / (df2.max() - df2.min())$. Guarda el resultado en `df_norm`. 3. Añade la columna 'Type 1' de vuelta al DataFrame normalizado: `df_norm['Type 1'] = df['Type 1']`. 4. Agrupa `df_norm` por 'Type 1' y calcula la media. Guarda en `agrupado_norm`. 5. Muestra `agrupado_norm` y verifica que todos los valores están entre 0 y 1.

```
[6]: # Podemos intentar NORMALIZAR nuestro dataset...
# Uso Min-Max porque para un radar necesito rango fijo y acotado, no Z-score
df2 = df[stats]
df_norm = (df2 - df2.min()) / (df2.max() - df2.min())
df_norm['Type 1'] = df['Type 1']

agrupado_norm = df_norm.groupby('Type 1').mean()
agrupado_norm
```

```
[6]:
```

	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed
Type 1						
Bug	0.383454	0.376175	0.282469	0.286086	0.246245	0.394665
Dark	0.461809	0.434392	0.276855	0.361625	0.253515	0.439017
Dragon	0.627996	0.625397	0.378959	0.624930	0.370748	0.578495
Electric	0.392420	0.327124	0.319836	0.581661	0.311204	0.512713

Fairy	0.463087	0.194444	0.348837	0.529412	0.428571	0.354839
Fighting	0.437200	0.476984	0.279734	0.426891	0.237415	0.537327
Fire	0.518736	0.456481	0.296318	0.561520	0.286508	0.463172
Flying	0.412752	0.222222	0.197674	0.358824	0.190476	0.541935
Ghost	0.471934	0.347475	0.293446	0.423797	0.240909	0.416129
Grass	0.461999	0.372973	0.261094	0.419078	0.263578	0.368439
Ground	0.530201	0.507310	0.331457	0.308359	0.226566	0.411885
Ice	0.508847	0.328283	0.215645	0.382353	0.233766	0.404692
Normal	0.469617	0.349850	0.197863	0.273132	0.179794	0.451613
Poison	0.454311	0.374786	0.258676	0.317195	0.225275	0.445658
Psychic	0.441540	0.413743	0.249204	0.518266	0.352632	0.527334
Rock	0.428955	0.445397	0.391362	0.348235	0.285170	0.366636
Steel	0.435937	0.462879	0.512051	0.343048	0.276407	0.326100
Water	0.515892	0.370755	0.288899	0.393452	0.269632	0.379550

1.5 Paso 4: Radar Chart Normalizado y Personalizado

Objetivo: Crear el gráfico radial definitivo con datos normalizados, eje fijo entre 0 y 1, y colores personalizados por tipo.

Instrucciones: 1. Extrae los valores normalizados de cada tipo usando `agrupado_norm.loc['Fire', :]` (igual que en el Paso 2 pero ahora de `agrupado_norm`). 2. Crea el `go.Figure()` y añade los tres `go.Scatterpolar`. Esta vez añade el parámetro `line_color` para asignar un color representativo a cada tipo (rojo para fuego, azul para agua, verde para planta). 3. En `fig.update_layout(...)`, configura el `radialaxis` con `range=[0, 1]` para que el eje esté fijo y sea comparable. 4. (Opcional): añade `opacity` a los trases para que los rellenos semitransparentes no se tapen entre sí. 5. Muestra el gráfico. ¡Ahora sí se pueden comparar los tres tipos claramente!

```
[7]: # Mismos tipos pero ya normalizados
fire2 = list(agrupado_norm.loc['Fire', :])
water2 = list(agrupado_norm.loc['Water', :])
grass2 = list(agrupado_norm.loc['Grass', :])

# Radar Chart con Plotly + colores intuitivos por tipo
# https://plotly.com/python/radar-chart/
fig = go.Figure()
fig.add_trace(go.Scatterpolar(
    r=fire2,
    theta=stats,
    fill='toself',
    name='Fire',
    line_color='red',      # rojo = fuego
    opacity=0.7           # transparencia para que se vean los solapes
))

fig.add_trace(go.Scatterpolar(
    r=water2,
```

```

        theta=stats,
        fill='toself',
        name='Water',
        line_color='blue',    # azul = agua
        opacity=0.7
    ))

fig.add_trace(go.Scatterpolar(
    r=grass2,
    theta=stats,
    fill='toself',
    name='Grass',
    line_color='green',    # verde = planta
    opacity=0.7
))

# Eje radial fijo en 0-1 para que la comparativa sea justa
fig.update_layout(
    title='Fire vs Water vs Grass (normalizado)',
    polar=dict(
        radialaxis=dict(visible=True, range=[0, 1])
    ),
    showlegend=True
)

fig.show()

```

Fire vs Water vs Grass (normalizado)



1.6 Reflexión Final del Alumno

Tómate un rato para reflexionar. * El **gráfico Radar Chart** es una herramienta muy potente para comparar perfiles multivariantes de forma visual. En este ejercicio hemos visto cómo el trío clásico de tipos (Fuego, Agua, Planta) tiene fortalezas y debilidades distintas: los Pokémon de tipo Fuego tienden a destacar en **Speed** y **Sp. Atk**, mientras que los de tipo Agua suelen ser más equilibrados. * Recuerda la lección más importante: **siempre normaliza los datos** antes de hacer un Radar Chart cuando las variables tienen escalas distintas. Sin normalización, el gráfico

puede ser visualmente engañoso o ilegible. * Como reto adicional: ¿podrías repetir el gráfico pero comparando **5 tipos** en lugar de 3? ¿O cambiar los tipos y ver qué tipos de Pokémon son más “redondos” (equilibrados en todas las estadísticas)?

Asegúrate de que cada celda tenga sus respectivos comentarios.

Metía Psychic y Dragon al trío Fire/Water/Grass. El más redondo me sale Water, las 6 stats le quedan parecidas sin tirar fuerte por ningún lado. Con más de 5 tipos ya no veo nada, se me solapan los rellenos.

```
[8]: # Reto: 5 tipos. Añado Psychic y Dragon al trío clásico
tipos_5 = ['Fire', 'Water', 'Grass', 'Psychic', 'Dragon']
colores_5 = ['red', 'blue', 'green', 'purple', 'orange']

fig = go.Figure()
for t, c in zip(tipos_5, colores_5):
    fig.add_trace(go.Scatterpolar(
        r=list(agrupado_norm.loc[t, :]),
        theta=stats,
        fill='toself',
        name=t,
        line_color=c,
        opacity=0.55          # bajo opacidad porque hay 5 capas que se solapan
    ))

fig.update_layout(
    title='Radar con 5 tipos (Fire/Water/Grass/Psychic/Dragon)',
    polar=dict(
        radialaxis=dict(visible=True, range=[0, 1])
    ),
    showlegend=True
)
fig.show()
```

Radar con 5 tipos (Fire/Water/Grass/Psychic/Dragon)



06_tarea_grafica_burbujas_happiness

April 27, 2026

1 Tarea paso a paso: Gráfico de Burbujas

¡ATENCIÓN! Lee esto antes de empezar: Esta tarea está diseñada paso a paso para que afiances los conceptos vistos en clase sobre los **Bubble Charts**. Su objetivo es que practiques cómo representar **tres o cuatro variables simultáneamente** en un único gráfico, usando la posición X, la posición Y, el tamaño de la burbuja y el color.

Nota importante sobre la Inteligencia Artificial: Es tentador copiar y pegar el enunciado de estos ejercicios en ChatGPT o Claude para que te den la solución... **Hacer eso perjudicará gravemente tu aprendizaje. (CODEA UN POCO QUE TE ENTERES QUE HACES).** La única forma de asimilar la sintaxis de las herramientas de visualización es escribiéndolas. **Usa tu cerebro, actúa paso a paso, equivócate, lee la documentación de Plotly, busca tu fallo y ¡aprende de él!**

1.1 Preparación y Datos

Para esta tarea vamos a utilizar el **World Happiness Report 2021**, un informe anual publicado por la ONU que mide el bienestar subjetivo de los ciudadanos de más de 150 países. Cada fila representa un país con indicadores como la puntuación de felicidad, el PIB per cápita, el soporte social o la esperanza de vida saludable.

Es un dataset ideal para un **Bubble Chart** porque permite cruzar variables económicas, sociales y de salud de todo el mundo en un solo vistazo.

1. Descarga el dataset de Kaggle: [World Happiness Report 2021](#)
2. Guarda el archivo `world-happiness-report-2021.csv` en la carpeta `data/` de tu proyecto.
3. En la siguiente celda, importa las librerías necesarias y carga el CSV en un DataFrame llamado `df`.

```
[1]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import plotly.express as px

# CARGA DE DATOS
df = pd.read_csv("data/world-happiness-report-2021.csv")

df.head(5)
```

```

[1]: Country name Regional indicator Ladder score \
0      Finland      Western Europe      7.842
1      Denmark      Western Europe      7.620
2      Switzerland   Western Europe      7.571
3      Iceland       Western Europe      7.554
4      Netherlands   Western Europe      7.464

Standard error of ladder score upperwhisker lowerwhisker \
0              0.032          7.904      7.780
1              0.035          7.687      7.552
2              0.036          7.643      7.500
3              0.059          7.670      7.438
4              0.027          7.518      7.410

Logged GDP per capita Social support Healthy life expectancy \
0          10.775          0.954          72.0
1          10.933          0.954          72.7
2          11.117          0.942          74.4
3          10.878          0.983          73.0
4          10.932          0.942          72.4

Freedom to make life choices Generosity Perceptions of corruption \
0              0.949          -0.098          0.186
1              0.946           0.030          0.179
2              0.919           0.025          0.292
3              0.955           0.160          0.673
4              0.913           0.175          0.338

Ladder score in Dystopia Explained by: Log GDP per capita \
0              2.43          1.446
1              2.43          1.502
2              2.43          1.566
3              2.43          1.482
4              2.43          1.501

Explained by: Social support Explained by: Healthy life expectancy \
0              1.106          0.741
1              1.108          0.763
2              1.079          0.816
3              1.172          0.772
4              1.079          0.753

Explained by: Freedom to make life choices Explained by: Generosity \
0              0.691          0.124
1              0.686          0.208
2              0.653          0.204
3              0.698          0.293

```

4	0.647	0.302
---	-------	-------

	Explained by: Perceptions of corruption	Dystopia + residual
0	0.481	3.253
1	0.485	2.868
2	0.413	2.839
3	0.170	2.967
4	0.384	2.798

```
[2]: # Ver la información del dataset
df.describe(include="all")
```

```
[2]: Country name  Regional indicator  Ladder score \
count          149          149    149.000000
unique          149           10         NaN
top            Finland  Sub-Saharan Africa         NaN
freq              1           36         NaN
mean            NaN          NaN    5.532839
std             NaN          NaN    1.073924
min             NaN          NaN    2.523000
25%             NaN          NaN    4.852000
50%             NaN          NaN    5.534000
75%             NaN          NaN    6.255000
max             NaN          NaN    7.842000
```

	Standard error of ladder score	upperwhisker	lowerwhisker	\
count	149.000000	149.000000	149.000000	
unique	NaN	NaN	NaN	
top	NaN	NaN	NaN	
freq	NaN	NaN	NaN	
mean	0.058752	5.648007	5.417631	
std	0.022001	1.054330	1.094879	
min	0.026000	2.596000	2.449000	
25%	0.043000	4.991000	4.706000	
50%	0.054000	5.625000	5.413000	
75%	0.070000	6.344000	6.128000	
max	0.173000	7.904000	7.780000	

	Logged GDP per capita	Social support	Healthy life expectancy	\
count	149.000000	149.000000	149.000000	
unique	NaN	NaN	NaN	
top	NaN	NaN	NaN	
freq	NaN	NaN	NaN	
mean	9.432208	0.814745	64.992799	
std	1.158601	0.114889	6.762043	
min	6.635000	0.463000	48.478000	
25%	8.541000	0.750000	59.802000	

50%	9.569000	0.832000	66.603000
75%	10.421000	0.905000	69.600000
max	11.647000	0.983000	76.953000

	Freedom to make life choices	Generosity	Perceptions of corruption \
count	149.000000	149.000000	149.000000
unique	NaN	NaN	NaN
top	NaN	NaN	NaN
freq	NaN	NaN	NaN
mean	0.791597	-0.015134	0.727450
std	0.113332	0.150657	0.179226
min	0.382000	-0.288000	0.082000
25%	0.718000	-0.126000	0.667000
50%	0.804000	-0.036000	0.781000
75%	0.877000	0.079000	0.845000
max	0.970000	0.542000	0.939000

	Ladder score in Dystopia	Explained by: Log GDP per capita \
count	1.490000e+02	149.000000
unique	NaN	NaN
top	NaN	NaN
freq	NaN	NaN
mean	2.430000e+00	0.977161
std	5.347044e-15	0.404740
min	2.430000e+00	0.000000
25%	2.430000e+00	0.666000
50%	2.430000e+00	1.025000
75%	2.430000e+00	1.323000
max	2.430000e+00	1.751000

	Explained by: Social support	Explained by: Healthy life expectancy \
count	149.000000	149.000000
unique	NaN	NaN
top	NaN	NaN
freq	NaN	NaN
mean	0.793315	0.520161
std	0.258871	0.213019
min	0.000000	0.000000
25%	0.647000	0.357000
50%	0.832000	0.571000
75%	0.996000	0.665000
max	1.172000	0.897000

	Explained by: Freedom to make life choices	Explained by: Generosity \
count	149.000000	149.000000
unique	NaN	NaN
top	NaN	NaN

freq	NaN	NaN
mean	0.498711	0.178047
std	0.137888	0.098270
min	0.000000	0.000000
25%	0.409000	0.105000
50%	0.514000	0.164000
75%	0.603000	0.239000
max	0.716000	0.541000

	Explained by: Perceptions of corruption	Dystopia + residual
count	149.000000	149.000000
unique	NaN	NaN
top	NaN	NaN
freq	NaN	NaN
mean	0.135141	2.430329
std	0.114361	0.537645
min	0.000000	0.648000
25%	0.060000	2.138000
50%	0.101000	2.509000
75%	0.174000	2.794000
max	0.547000	3.482000

```
[3]: # Limpieza de nulos
df = df.dropna()
df
```

```
[3]: Country name Regional indicator Ladder score \
0 Finland Western Europe 7.842
1 Denmark Western Europe 7.620
2 Switzerland Western Europe 7.571
3 Iceland Western Europe 7.554
4 Netherlands Western Europe 7.464
.. ...
144 Lesotho Sub-Saharan Africa 3.512
145 Botswana Sub-Saharan Africa 3.467
146 Rwanda Sub-Saharan Africa 3.415
147 Zimbabwe Sub-Saharan Africa 3.145
148 Afghanistan South Asia 2.523
```

	Standard error of ladder score	upperwhisker	lowerwhisker	\
0	0.032	7.904	7.780	
1	0.035	7.687	7.552	
2	0.036	7.643	7.500	
3	0.059	7.670	7.438	
4	0.027	7.518	7.410	
..	...	...	...	
144	0.120	3.748	3.276	

145	0.074	3.611	3.322
146	0.068	3.548	3.282
147	0.058	3.259	3.030
148	0.038	2.596	2.449

	Logged GDP per capita	Social support	Healthy life expectancy \
0	10.775	0.954	72.000
1	10.933	0.954	72.700
2	11.117	0.942	74.400
3	10.878	0.983	73.000
4	10.932	0.942	72.400
..	...	...	...
144	7.926	0.787	48.700
145	9.782	0.784	59.269
146	7.676	0.552	61.400
147	7.943	0.750	56.201
148	7.695	0.463	52.493

	Freedom to make life choices	Generosity	Perceptions of corruption \
0	0.949	-0.098	0.186
1	0.946	0.030	0.179
2	0.919	0.025	0.292
3	0.955	0.160	0.673
4	0.913	0.175	0.338
..	...	...	...
144	0.715	-0.131	0.915
145	0.824	-0.246	0.801
146	0.897	0.061	0.167
147	0.677	-0.047	0.821
148	0.382	-0.102	0.924

	Ladder score in Dystopia	Explained by: Log GDP per capita \
0	2.43	1.446
1	2.43	1.502
2	2.43	1.566
3	2.43	1.482
4	2.43	1.501
..	...	...
144	2.43	0.451
145	2.43	1.099
146	2.43	0.364
147	2.43	0.457
148	2.43	0.370

	Explained by: Social support	Explained by: Healthy life expectancy \
0	1.106	0.741
1	1.108	0.763

2	1.079	0.816
3	1.172	0.772
4	1.079	0.753
..	...	...
144	0.731	0.007
145	0.724	0.340
146	0.202	0.407
147	0.649	0.243
148	0.000	0.126

	Explained by: Freedom to make life choices	Explained by: Generosity \
0	0.691	0.124
1	0.686	0.208
2	0.653	0.204
3	0.698	0.293
4	0.647	0.302
..	...	...
144	0.405	0.103
145	0.539	0.027
146	0.627	0.227
147	0.359	0.157
148	0.000	0.122

	Explained by: Perceptions of corruption	Dystopia + residual
0	0.481	3.253
1	0.485	2.868
2	0.413	2.839
3	0.170	2.967
4	0.384	2.798
..	...	...
144	0.015	1.800
145	0.088	0.648
146	0.493	1.095
147	0.075	1.205
148	0.010	1.895

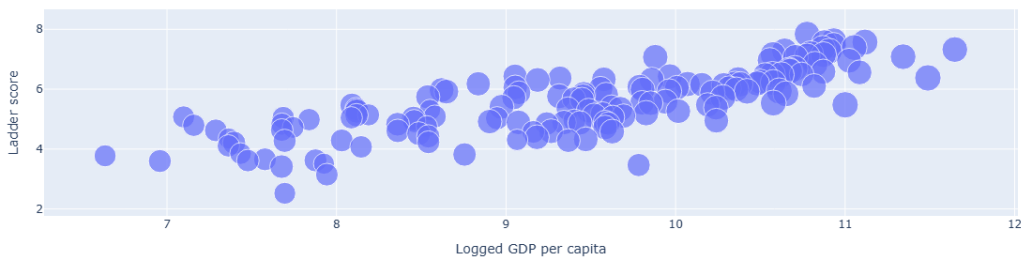
[149 rows x 20 columns]

1.2 Paso 1: Bubble Chart Básico (Tres Dimensiones)

Objetivo: Crear un gráfico de burbujas básico que represente tres variables numéricas simultáneamente: dos como posición (ejes X e Y) y una tercera como el tamaño de cada burbuja.

Instrucciones: 1. Usa `px.scatter()` de Plotly Express. 2. Asigna al eje **X** la columna 'Logged GDP per capita' (riqueza del país). 3. Asigna al eje **Y** la columna 'Ladder score' (puntuación de felicidad, de 0 a 10). 4. Asigna al parámetro `size` la columna 'Healthy life expectancy' (esperanza de vida saludable). Esto controla el **tamaño** de cada burbuja. 5. Muestra el gráfico con `fig.show()` y observa el resultado. ¿Se ve ya alguna tendencia entre riqueza y felicidad?


```
[4]: # Bubble Charts en Plotly
# https://plotly.com/python/bubble-charts/
fig = px.scatter(
    df,                                     # DataFrame del Happiness Report 2021
    x='Logged GDP per capita',              # eje X: PIB per cápita en log
    y='Ladder score',                      # eje Y: felicidad (0-10)
    size='Healthy life expectancy',        # tamaño: esperanza de vida sana
)
fig.show()
```

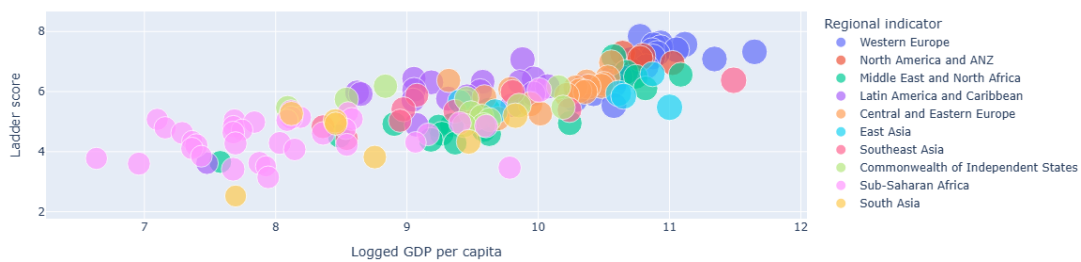


1.3 Paso 2: Añadir una Cuarta Dimensión con Color

Objetivo: Incorporar una variable categórica al gráfico usando el color de las burbujas. Esto nos permitirá ver si los países agrupados por región mundial siguen patrones distintos.

Instrucciones: 1. Repite el gráfico del Paso 1. 2. Añade el parámetro `color` y asígnale la columna `'Regional indicator'` (región del mundo a la que pertenece el país). 3. Añade también el parámetro `hover_name='Country name'` para que al pasar el ratón por encima de una burbuja se muestre el nombre del país. 4. Observa cómo el color revela agrupaciones regionales claras. ¿Qué regiones tienden a tener mayor felicidad? ¿Y mayor PIB?

```
[5]: # Igual que el paso 1 pero añadiendo color por región y hover con el país
fig = px.scatter(
    df,
    x='Logged GDP per capita',
    y='Ladder score',
    size='Healthy life expectancy',
    color='Regional indicator',            # color por región
    hover_name='Country name',           # país al pasar el ratón
)
fig.show()
```



1.4 Paso 3: Versión con Seaborn

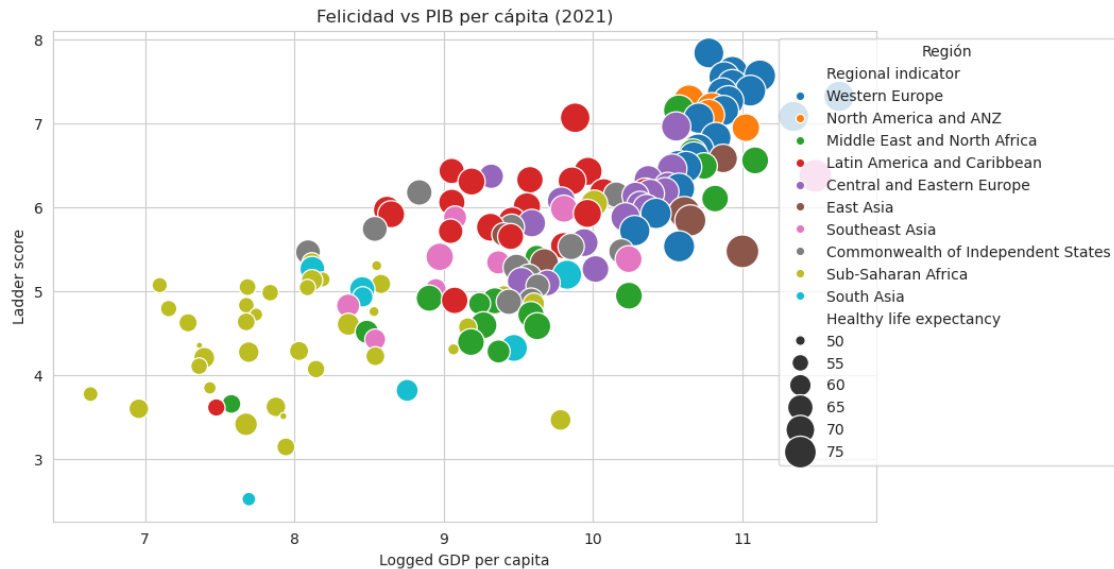
Objetivo: Reproducir el mismo gráfico de burbujas usando la librería Seaborn + Matplotlib. Aprenderás que Seaborn no tiene un tipo de gráfico “bubble” específico, sino que se consigue con `sns.scatterplot` controlando manualmente el parámetro `size`.

Instrucciones: 1. Usa `sns.scatterplot()` con el mismo DataFrame `df`. 2. Asigna `x`, `y` y `hue` (equivalente al color de Plotly) igual que en el Paso 2. 3. Para el tamaño, usa el parámetro `size='Healthy life expectancy'` y añade `sizes=(20, 500)` para controlar el rango de tamaños mínimo y máximo en píxeles. 4. Añade un título con `plt.title(...)` y muestra el gráfico con `plt.show()`. 5. Compara la experiencia con Plotly: ¿cuál te parece más interactivo y útil para explorar datos?

```
[6]: # Mismo gráfico en Seaborn (sns.scatterplot con size manual)
sns.set_style("whitegrid")

plt.figure(figsize=(10, 6))
sns.scatterplot(
    data=df,
    x='Logged GDP per capita',
    y='Ladder score',
    size='Healthy life expectancy',
    sizes=(20, 500),                # rango en píxeles para que se note la
    ↪diferencia
    hue='Regional indicator',       # hue = color en Plotly
    legend=True,
)

plt.title('Felicidad vs PIB per cápita (2021)')
# Saco la leyenda fuera para que no tape burbujas
plt.legend(bbox_to_anchor=(1.3, 1), loc='upper right', title='Región')
plt.show()
```



1.5 Paso 4: Personalización del Gráfico Plotly

Objetivo: Pulir el gráfico de Plotly con títulos descriptivos, etiquetas de ejes, control del tamaño máximo de las burbujas y datos adicionales en el tooltip (hover).

Instrucciones: 1. Parte del gráfico del Paso 2 y añade los siguientes parámetros a `px.scatter()`:
 - `size_max=50` para controlar el tamaño máximo de las burbujas. - `hover_data=['Social support', 'Freedom to make life choices']` para mostrar datos extra en el tooltip. - `title` con un título descriptivo para el gráfico. - `labels` con un diccionario que renombre las columnas en el gráfico (por ejemplo: `{ 'Logged GDP per capita': 'PIB per cápita (log)', 'Ladder score': 'Felicidad' }`). 2. Usa `fig.update_layout(legend_title_text='Región')` para cambiar el título de la leyenda. 3. Muestra el gráfico final.

```
[7]: # Plotly final: título, labels en español, size_max y hover extra
fig = px.scatter(
    df,
    x='Logged GDP per capita',
    y='Ladder score',
    size='Healthy life expectancy',
    color='Regional indicator',
    hover_name='Country name',
    size_max=50, # tamaño
    # máx de las burbujas
    hover_data=['Social support', 'Freedom to make life choices'], # datos
    # extra al pasar el ratón
    title='World Happiness Report 2021 - PIB, felicidad y esperanza de vida',
    labels={
        'Logged GDP per capita': 'PIB per cápita (log)',
```

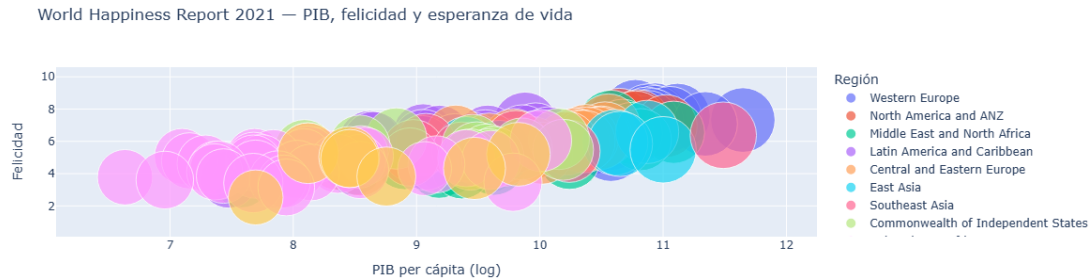
```

        'Ladder score': 'Felicidad',
        'Healthy life expectancy': 'Esperanza de vida sana',
    },
)

fig.update_layout(legend_title_text='Región')

fig.show()

```



1.5.1 Interpretación del Gráfico de Burbujas

Este gráfico final nos permite analizar simultáneamente **cuatro dimensiones** de nuestros datos en una sola imagen:

1. **La posición Horizontal (PIB per cápita - Logged GDP per capita):** Representa la riqueza del país. Al estar en escala logarítmica, las diferencias entre países pobres y ricos se perciben mejor.
2. **La posición Vertical (Felicidad - Ladder score):** Mide el bienestar subjetivo de los ciudadanos en una escala de 0 a 10.
3. **El Tamaño de la Burbuja (Esperanza de vida - Healthy life expectancy):** Los países con burbujas más grandes tienen ciudadanos más sanos y longevos.
4. **El Color (Región - Regional indicator):** Permite identificar de un vistazo qué regiones del mundo agrupan a los países más felices o más ricos.

Conclusión principal: existe una **correlación positiva clara** entre el PIB per cápita y la felicidad: a mayor riqueza, mayor bienestar. Los países de Europa Occidental y Norteamérica dominan la esquina superior derecha del gráfico.

1.6 Reflexión Final del Alumno

Tómate un rato para reflexionar. * El **Bubble Chart** es una extensión natural del scatter plot que permite añadir hasta cuatro dimensiones a un gráfico 2D. Es especialmente útil cuando tienes datos con un identificador claro por punto (un país, una empresa, una persona) y quieres comparar múltiples métricas de golpe. * Recuerda su principal limitación: si tienes demasiados puntos o las burbujas son muy similares en tamaño, el gráfico se vuelve difícil de leer. En esos casos, filtra los datos o agrupa por categorías antes de visualizar. * Como reto adicional: ¿podrías

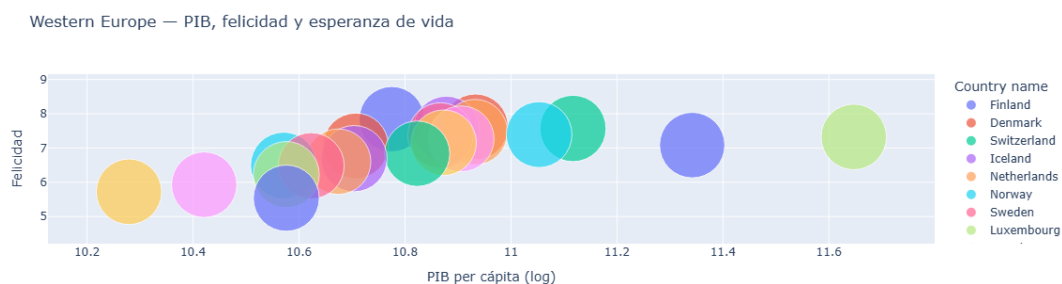
filtrar el DataFrame para quedarte solo con una región (por ejemplo, 'Western Europe') y hacer un gráfico de burbujas solo con esos países? ¿O cambiar la variable del tamaño a 'Generosity' y ver qué cambia?

Asegúrate de que cada celda tenga sus respectivos comentarios.

Filtrando a Western Europe con `df[df['Regional indicator']=='Western Europe']` se verían diferencias dentro del bloque rico que ahora se comen los países pobres. Cambiar size a Generosity rompe el patrón, la generosidad no va pegada al PIB.

```
[8]: # Reto: solo Western Europe (para ver diferencias dentro del bloque rico que en
    ↪ el global se pierden)
df_we = df[df['Regional indicator'] == 'Western Europe']

fig = px.scatter(
    df_we,
    x='Logged GDP per capita',
    y='Ladder score',
    size='Healthy life expectancy',
    color='Country name',          # cambio color a país, ya que todos son
    ↪ la misma región
    hover_name='Country name',
    size_max=50,
    title='Western Europe - PIB, felicidad y esperanza de vida',
    labels={
        'Logged GDP per capita': 'PIB per cápita (log)',
        'Ladder score': 'Felicidad',
    },
)
fig.show()
```



07_tarea_grafico_lineas_dobles_avocado

April 27, 2026

1 Tarea paso a paso: Gráfico de Líneas con Doble Eje Y

¡ATENCIÓN! Lee esto antes de empezar: Esta tarea está diseñada paso a paso para que afiances los conceptos vistos en clase sobre los **Dual-Axis Line Charts**. Su objetivo es que practiques cómo representar **dos series temporales con escalas muy distintas** en un único gráfico compartiendo el mismo eje X.

Nota importante sobre la Inteligencia Artificial: Es tentador copiar y pegar el enunciado de estos ejercicios en ChatGPT o Claude para que te den la solución... **Hacer eso perjudicará gravemente tu aprendizaje. (CODEA UN POCO QUE TE ENTERES QUE HACES).** La única forma de asimilar la sintaxis de las herramientas de visualización es escribiéndolas. **Usa tu cerebro, actúa paso a paso, equivócate, lee la documentación de Matplotlib y Plotly, busca tu fallo y ¡aprende de él!**

1.1 Preparación y Datos

Para esta tarea vamos a utilizar el dataset “**Avocado Prices**”, que recoge el precio medio semanal y el volumen total de aguacates vendidos en distintas ciudades de Estados Unidos entre 2015 y 2018.

Es un dataset ideal para un **Dual-Axis Line Chart** porque las dos variables de interés tienen escalas completamente distintas: - **AveragePrice**: precio medio de un aguacate, en dólares (~0.5 \$ a ~3 \$). - **Total Volume**: número total de aguacates vendidos (~1.000 a ~60.000.000).

Si las pusiésemos en el mismo eje Y, la línea del precio quedaría completamente aplastada y sería ilegible.

1. Descarga el dataset de Kaggle: [Avocado Prices](#)
2. Guarda el archivo `avocado.csv` en la carpeta `data/` de tu proyecto.
3. En la siguiente celda, importa las librerías necesarias y carga el CSV en un DataFrame llamado `df`.

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import plotly.graph_objects as go
from mpl_toolkits.axes_grid1 import host_subplot

# CARGA DE DATOS
df = pd.read_csv("data/avocado.csv")
```

```
df.head(5)
```

```
[1]: Unnamed: 0      Date AveragePrice Total Volume      4046      4225 \
0          0  2015-12-27          1.33      64236.62  1036.74  54454.85
1          1  2015-12-20          1.35      54876.98   674.28  44638.81
2          2  2015-12-13          0.93     118220.22   794.70 109149.67
3          3  2015-12-06          1.08      78992.15  1132.00   71976.41
4          4  2015-11-29          1.28      51039.60   941.48   43838.39
```

```
      4770 Total Bags Small Bags Large Bags XLarge Bags      type \
0   48.16    8696.87    8603.62      93.25         0.0 conventional
1   58.33    9505.56    9408.07      97.49         0.0 conventional
2  130.50    8145.35    8042.21     103.14         0.0 conventional
3   72.58    5811.16    5677.40     133.76         0.0 conventional
4   75.78    6183.95    5986.26     197.69         0.0 conventional
```

```
      year region
0   2015  Albany
1   2015  Albany
2   2015  Albany
3   2015  Albany
4   2015  Albany
```

```
[2]: # Ver la información del dataset
df.describe(include="all")
```

```
[2]: Unnamed: 0      Date AveragePrice Total Volume      4046 \
count    18249.000000    18249  18249.000000  1.824900e+04  1.824900e+04
unique           NaN      169           NaN           NaN           NaN
top           NaN  2015-12-27           NaN           NaN           NaN
freq           NaN      108           NaN           NaN           NaN
mean        24.232232      NaN      1.405978  8.506440e+05  2.930084e+05
std         15.481045      NaN      0.402677  3.453545e+06  1.264989e+06
min           0.000000      NaN      0.440000  8.456000e+01  0.000000e+00
25%          10.000000      NaN      1.100000  1.083858e+04  8.540700e+02
50%          24.000000      NaN      1.370000  1.073768e+05  8.645300e+03
75%          38.000000      NaN      1.660000  4.329623e+05  1.110202e+05
max          52.000000      NaN      3.250000  6.250565e+07  2.274362e+07
```

```
      4225      4770 Total Bags Small Bags Large Bags \
count  1.824900e+04  1.824900e+04  1.824900e+04  1.824900e+04  1.824900e+04
unique           NaN           NaN           NaN           NaN           NaN
top           NaN           NaN           NaN           NaN           NaN
freq           NaN           NaN           NaN           NaN           NaN
mean  2.951546e+05  2.283974e+04  2.396392e+05  1.821947e+05  5.433809e+04
std   1.204120e+06  1.074641e+05  9.862424e+05  7.461785e+05  2.439660e+05
```

min	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00
25%	3.008780e+03	0.000000e+00	5.088640e+03	2.849420e+03	1.274700e+02
50%	2.906102e+04	1.849900e+02	3.974383e+04	2.636282e+04	2.647710e+03
75%	1.502069e+05	6.243420e+03	1.107834e+05	8.333767e+04	2.202925e+04
max	2.047057e+07	2.546439e+06	1.937313e+07	1.338459e+07	5.719097e+06

	XLarge Bags	type	year	region
count	18249.000000	18249	18249.000000	18249
unique	NaN	2	NaN	54
top	NaN	conventional	NaN	Albany
freq	NaN	9126	NaN	338
mean	3106.426507	NaN	2016.147899	NaN
std	17692.894652	NaN	0.939938	NaN
min	0.000000	NaN	2015.000000	NaN
25%	0.000000	NaN	2015.000000	NaN
50%	0.000000	NaN	2016.000000	NaN
75%	132.500000	NaN	2017.000000	NaN
max	551693.650000	NaN	2018.000000	NaN

```
[3]: # Limpieza de nulos
df = df.dropna()
df
```

```
[3]:      Unnamed: 0      Date  AveragePrice  Total Volume      4046      4225  \
0              0  2015-12-27           1.33      64236.62  1036.74  54454.85
1              1  2015-12-20           1.35      54876.98   674.28  44638.81
2              2  2015-12-13           0.93     118220.22   794.70  109149.67
3              3  2015-12-06           1.08      78992.15  1132.00   71976.41
4              4  2015-11-29           1.28      51039.60   941.48   43838.39
...           ...      ...      ...      ...      ...      ...
18244          7  2018-02-04           1.63      17074.83  2046.96   1529.20
18245          8  2018-01-28           1.71      13888.04  1191.70   3431.50
18246          9  2018-01-21           1.87      13766.76  1191.92   2452.79
18247         10  2018-01-14           1.93      16205.22  1527.63   2981.04
18248         11  2018-01-07           1.62      17489.58  2894.77   2356.13
```

	4770	Total Bags	Small Bags	Large Bags	XLarge Bags	type
0	48.16	8696.87	8603.62	93.25	0.0	conventional
1	58.33	9505.56	9408.07	97.49	0.0	conventional
2	130.50	8145.35	8042.21	103.14	0.0	conventional
3	72.58	5811.16	5677.40	133.76	0.0	conventional
4	75.78	6183.95	5986.26	197.69	0.0	conventional
...	...	...	...	...	...	...
18244	0.00	13498.67	13066.82	431.85	0.0	organic
18245	0.00	9264.84	8940.04	324.80	0.0	organic
18246	727.94	9394.11	9351.80	42.31	0.0	organic
18247	727.01	10969.54	10919.54	50.00	0.0	organic


```
18248  224.53      12014.15    11988.14      26.01      0.0      organic
```

```
      year      region
0      2015      Albany
1      2015      Albany
2      2015      Albany
3      2015      Albany
4      2015      Albany
...
18244  2018  WestTexNewMexico
18245  2018  WestTexNewMexico
18246  2018  WestTexNewMexico
18247  2018  WestTexNewMexico
18248  2018  WestTexNewMexico
```

```
[18249 rows x 14 columns]
```

1.2 Paso 1: Filtrar y Preparar los Datos

Objetivo: Obtener una serie temporal limpia filtrando el dataset a un único mercado y tipo de aguacate, y agrupando por año.

Instrucciones: 1. Filtra el DataFrame para quedarte solo con las filas donde `region == 'TotalUS'` y `type == 'conventional'`. Guarda el resultado en `df_us`. 2. Agrupa `df_us` por la columna `year` y calcula: - La **media** de `AveragePrice`. - La **suma** de `Total Volume`.

Usa `.agg({'AveragePrice': 'mean', 'Total Volume': 'sum'})` y guarda en `df_agrupado`. 3. Muestra `df_agrupado` para verificar que tienes 4 filas (una por año: 2015, 2016, 2017, 2018).

```
[4]: # filtramos region TotalUS y aguacate convencional, agrupamos por año
# mean en precio porque se promedia a lo largo del año, sum en volumen porque
      ↪ se acumula
df_us = df[(df['region'] == 'TotalUS') & (df['type'] == 'conventional')]
df_agrupado = df_us.groupby('year').agg({
    'AveragePrice': 'mean',
    'Total Volume': 'sum'
})
df_agrupado
```

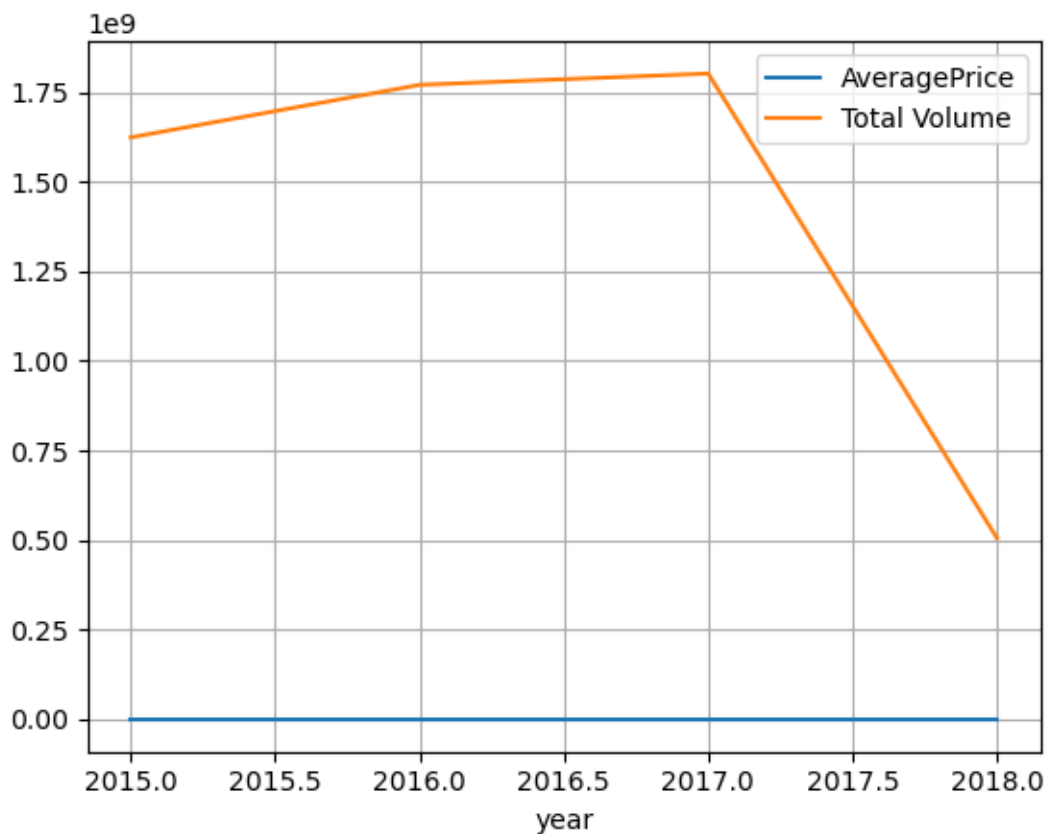
```
[4]:      AveragePrice  Total Volume
year
2015      1.012500  1.623686e+09
2016      1.046731  1.770259e+09
2017      1.221698  1.801770e+09
2018      1.060000  5.055064e+08
```

1.3 Paso 2: El Problema del Eje Único

Objetivo: Ver con tus propios ojos por qué necesitamos un doble eje Y cuando las variables tienen escalas muy distintas.

Instrucciones: 1. Usando `df_agrupado`, dibuja un gráfico de líneas con `df_agrupado.plot.line()` que muestre **las dos variables** (`AveragePrice` y `Total Volume`) en el **mismo eje Y**. Usa el parámetro `y=['AveragePrice', 'Total Volume']`. 2. Activa la cuadrícula con `grid=True`. 3. Muestra el gráfico. ¿Qué le ocurre a la línea de `AveragePrice`? ¿Por qué?

```
[5]: # Vemos como se queda si graficamos todo en un mismo eje, perdemos noción de la
      ↪escala
df_agrupado.plot.line(
    y=['AveragePrice', 'Total Volume'],
    grid=True
)
plt.show()
```



1.3.1 Interpretación

La línea de `AveragePrice` queda completamente aplastada contra el eje X porque sus valores (entre 1 y 2) son insignificantes comparados con los de `Total Volume` (en el orden de los miles de millones). Necesitamos un segundo eje Y con su propia escala.

1.4 Paso 3: Doble Eje Y con Matplotlib (`twinx`)

Objetivo: Crear el gráfico de doble eje Y usando el método `twinx()` de Matplotlib, que crea un segundo eje Y que comparte el mismo eje X.

Instrucciones: 1. Crea la figura con `fig, ax1 = plt.subplots(figsize=(10, 5))`. 2. Dibuja la primera línea en `ax1`: usa `ax1.plot(...)` con el índice de `df_agrupado` como X y `df_agrupado['AveragePrice']` como Y. Ponle un color (por ejemplo, `'tab:green'`) y una etiqueta. 3. Crea el segundo eje con `ax2 = ax1.twinx()`. Dibuja la segunda línea en `ax2` con `ax2.plot(...)` usando `df_agrupado['Total Volume']` como Y y un color distinto (por ejemplo, `'tab:orange'`). 4. Etiqueta los ejes Y de cada eje con `ax1.set_ylabel(...)` y `ax2.set_ylabel(...)`, usando el mismo color que su línea para que quede claro cuál eje corresponde a qué variable. 5. Añade un título con `plt.title(...)` y muestra el gráfico.

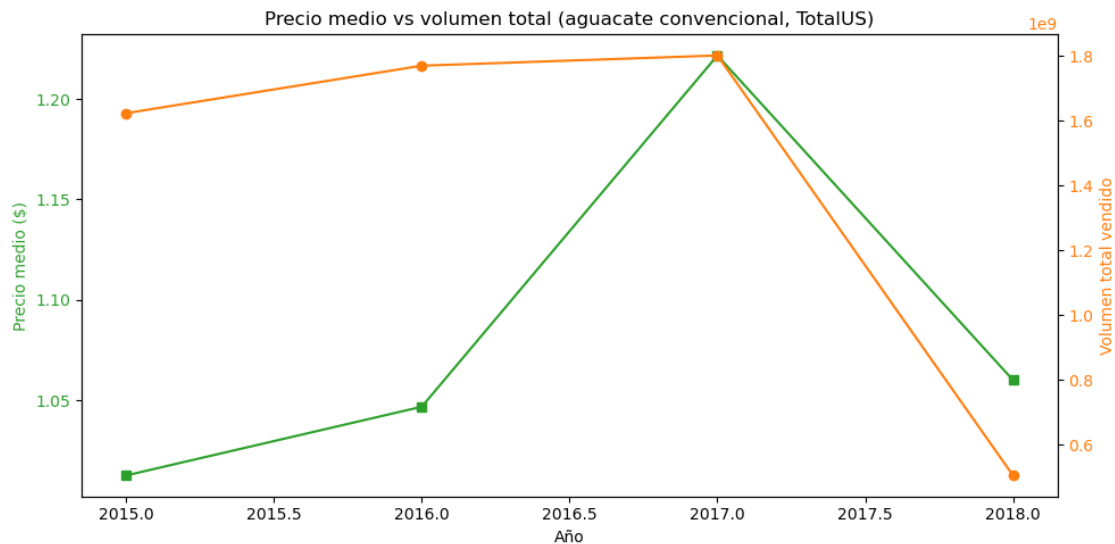
```
[6]: # Creación de un gráfico con dos ejes Y utilizando Matplotlib
# https://matplotlib.org/stable/gallery/subplots_axes_and_figures/two_scales.
# ↪html
fig, ax1 = plt.subplots(figsize=(10, 5))

ax1.set_xlabel('Año')
ax1.set_ylabel('Precio medio ($)', color='tab:green') # verde por aguacate
ax1.plot(
    df_agrupado.index.values,
    df_agrupado['AveragePrice'],
    color='tab:green',
    marker='s'
)
ax1.tick_params(axis='y', labelcolor='tab:green')

# Segundo eje Y compartiendo X
ax2 = ax1.twinx()
ax2.set_ylabel('Volumen total vendido', color='tab:orange') # naranja para
# ↪distinguir
ax2.plot(
    df_agrupado.index.values,
    df_agrupado['Total Volume'],
    color='tab:orange',
    marker='o'
)
ax2.tick_params(axis='y', labelcolor='tab:orange')

plt.title('Precio medio vs volumen total (aguacate convencional, TotalUS)')
fig.tight_layout()
```

```
plt.show()
```



1.5 Paso 4: Doble Eje Y con Plotly

Objetivo: Reproducir el mismo gráfico de doble eje Y usando Plotly, que ofrece interactividad (zoom, hover, toggle de series) sin esfuerzo extra.

Instrucciones: 1. Crea una figura vacía con `fig = go.Figure()`. 2. Añade la primera serie con `fig.add_trace(go.Scatter(...))`. Usa el índice de `df_agrupado` como `x`, `df_agrupado['AveragePrice']` como `y`, `name='Precio Medio ($)'`, `mode='markers+lines'` y `marker=dict(color='green')`. 3. Añade la segunda serie con otro `fig.add_trace(go.Scatter(...))`. Esta vez añade `yaxis='y2'` para indicar que usa el eje Y secundario. Usa un color distinto. 4. Configura el layout con `fig.update_layout(...)` incluyendo: - `title`: un título descriptivo. - `yaxis`: diccionario con `title='Precio Medio ($)'`. - `yaxis2`: diccionario con `title='Volumen Total de Ventas'`, `overlaying='y'` y `side='right'`. 5. Muestra el gráfico con `fig.show()`.

```
[7]: # Grafico de lineas con doble eje Y en Plotly
fig = go.Figure()

# Primera serie: precio en el eje Y principal
fig.add_trace(
    go.Scatter(
        x=df_agrupado.index.values,
        y=df_agrupado['AveragePrice'],
        name='Precio Medio ($)',
        marker=dict(color='green'),
        mode='markers+lines'
    )
)
```

```

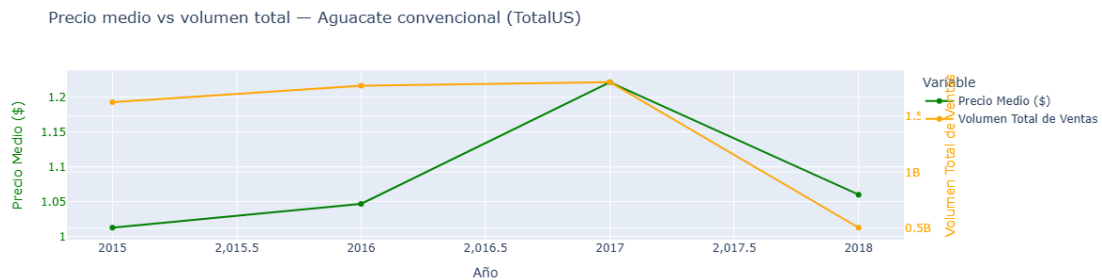
)

# Segunda serie: volumen en el eje Y secundario (yaxis='y2' es la clave)
fig.add_trace(
    go.Scatter(
        x=df_agrupado.index.values,
        y=df_agrupado['Total Volume'],
        name='Volumen Total de Ventas',
        marker=dict(color='orange'),
        mode='markers+lines',
        yaxis='y2'
    )
)

fig.update_layout(
    title='Precio medio vs volumen total - Aguacate convencional (TotalUS)',
    xaxis_title='Año',
    yaxis=dict(
        title=dict(text='Precio Medio ($)', font=dict(color='green')),
        tickfont=dict(color='green')
    ),
    yaxis2=dict(
        title=dict(text='Volumen Total de Ventas', font=dict(color='orange')),
        tickfont=dict(color='orange'),
        overlaying='y',          # superponer al eje Y principal
        side='right'             # colocar el eje secundario a la derecha
    ),
    legend_title='Variable'
)

fig.show()

```



1.5.1 Interpretación del Gráfico de Doble Eje

Al usar un **doble eje Y**, cada variable tiene su propia escala y ambas líneas son perfectamente legibles. Ahora podemos extraer conclusiones reales:

1. **Eje Y izquierdo (Precio):** muestra la evolución del precio medio del aguacate en dólares a lo largo de los años.
2. **Eje Y derecho (Volumen):** muestra el total de aguacates vendidos en EE. UU., una magnitud millones de veces mayor.
3. **Conclusión:** ¿Sube el precio cuando baja el volumen vendido, o viceversa? El gráfico te permite responder esta pregunta de un vistazo, algo imposible con un eje único.

1.6 Reflexión Final del Alumno

Tómate un rato para reflexionar. * El **gráfico de doble eje Y** resuelve un problema muy concreto: comparar la evolución temporal de dos variables con escalas incompatibles. Sin él, una de las dos series quedaría invisible o distorsionada. * Recuerda su principal riesgo: este tipo de gráfico puede ser **engañoso** si no se etiquetan correctamente los ejes o si se usan para sugerir correlaciones que no existen. Úsalo con honestidad y siempre indica claramente qué eje corresponde a qué variable. * Como reto adicional: ¿podrías repetir el análisis pero filtrando por `type == 'organic'`? ¿El comportamiento del precio y el volumen es diferente al de los aguacates convencionales?

Asegúrate de que cada celda tenga sus respectivos comentarios.

El precio se dispara y el volumen se hunde, el orgánico es un nicho. La forma de la curva yo creo que es la misma pero a otra escala.

```
[8]: # Reto: mismo análisis con aguacate orgánico (espero precio más alto, volumen
      mucho menor)
df_us_org = df[(df['region'] == 'TotalUS') & (df['type'] == 'organic')]
df_agrupado_org = df_us_org.groupby('year').agg({
    'AveragePrice': 'mean',
    'Total Volume': 'sum'
})

fig, ax1 = plt.subplots(figsize=(10, 5))

ax1.set_xlabel('Año')
ax1.set_ylabel('Precio medio ($)', color='tab:green')
ax1.plot(
    df_agrupado_org.index.values,
    df_agrupado_org['AveragePrice'],
    color='tab:green',
    marker='s'
)
ax1.tick_params(axis='y', labelcolor='tab:green')

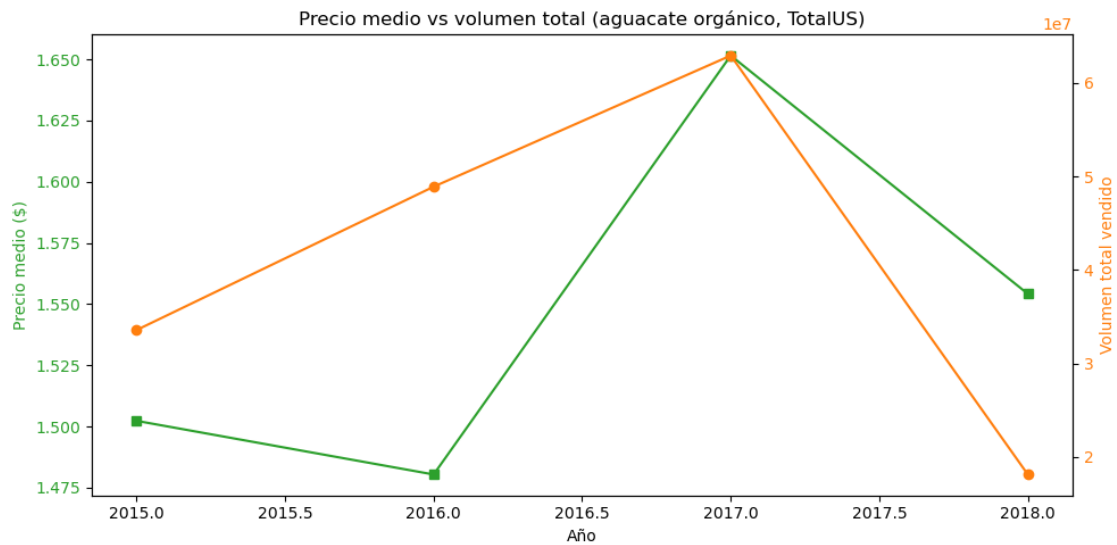
ax2 = ax1.twinx()
ax2.set_ylabel('Volumen total vendido', color='tab:orange')
ax2.plot(
```

```

df_agrupado_org.index.values,
df_agrupado_org['Total Volume'],
color='tab:orange',
marker='o'
)
ax2.tick_params(axis='y', labelcolor='tab:orange')

plt.title('Precio medio vs volumen total (aguacate orgánico, TotalUS)')
fig.tight_layout()
plt.show()

```



08_tarea_grafico_barras_lineas_superstore

April 27, 2026

1 Tarea paso a paso: Gráfico Combinado Barras y Líneas

¡ATENCIÓN! Lee esto antes de empezar: Esta tarea está diseñada paso a paso para que afiances los conceptos vistos en clase sobre los **Bar + Line Charts**. Su objetivo es que practiques cómo combinar un gráfico de barras y una línea en el mismo gráfico, usando un eje Y secundario cuando las escalas de las variables son muy distintas.

Nota importante sobre la Inteligencia Artificial: Es tentador copiar y pegar el enunciado de estos ejercicios en ChatGPT o Claude para que te den la solución... **Hacer eso perjudicará gravemente tu aprendizaje. (CODEA UN POCO QUE TE ENTERES QUE HACES).** La única forma de asimilar la sintaxis de las herramientas de visualización es escribiéndolas. **Usa tu cerebro, actúa paso a paso, equivócate, lee la documentación de Seaborn y Plotly, busca tu fallo y ¡aprende de él!**

1.1 Preparación y Datos

Para esta tarea vamos a utilizar el dataset **“Superstore Sales”**, un dataset clásico de Business Intelligence que recoge las ventas de una tienda americana de 2014 a 2017. Cada fila representa un pedido con su categoría de producto, ventas totales, beneficio y región.

Es un dataset ideal para un **Bar + Line Chart** porque permite combinar dos magnitudes relacionadas pero con escalas muy distintas: - **Sales:** ingresos totales por ventas (en dólares, orden de magnitud: miles a millones). - **Profit:** beneficio neto obtenido, que puede ser negativo si hay pérdidas (mucho menor en escala que Sales).

Las barras representarán el volumen de ventas (¿cuánto vendemos?) y la línea el beneficio (¿cuánto ganamos?), que es el gráfico más usado en informes de negocio.

1. Descarga el dataset de Kaggle: [Superstore Sales Dataset](#)
2. Guarda el archivo como `superstore.csv` en la carpeta `data/` de tu proyecto.
3. En la siguiente celda, importa las librerías necesarias y carga el CSV en un DataFrame llamado `df`.

```
[1]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import plotly.graph_objects as go
from plotly.subplots import make_subplots
```



```
# CARGA DE DATOS
df = pd.read_csv("data/Sample - Superstore.csv", encoding='latin-1')

df.head(5)
```

```
[1]:
```

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID \
0	1	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520
1	2	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520
2	3	CA-2016-138688	6/12/2016	6/16/2016	Second Class	DV-13045
3	4	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335
4	5	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335

	Customer Name	Segment	Country	City ... \
0	Claire Gute	Consumer	United States	Henderson ...
1	Claire Gute	Consumer	United States	Henderson ...
2	Darrin Van Huff	Corporate	United States	Los Angeles ...
3	Sean O'Donnell	Consumer	United States	Fort Lauderdale ...
4	Sean O'Donnell	Consumer	United States	Fort Lauderdale ...

	Postal Code	Region	Product ID	Category	Sub-Category \
0	42420	South	FUR-BO-10001798	Furniture	Bookcases
1	42420	South	FUR-CH-10000454	Furniture	Chairs
2	90036	West	OFF-LA-10000240	Office Supplies	Labels
3	33311	South	FUR-TA-10000577	Furniture	Tables
4	33311	South	OFF-ST-10000760	Office Supplies	Storage

	Product Name	Sales	Quantity \
0	Bush Somerset Collection Bookcase	261.9600	2
1	Hon Deluxe Fabric Upholstered Stacking Chairs,...	731.9400	3
2	Self-Adhesive Address Labels for Typewriters b...	14.6200	2
3	Bretford CR4500 Series Slim Rectangular Table	957.5775	5
4	Eldon Fold 'N Roll Cart System	22.3680	2

	Discount	Profit
0	0.00	41.9136
1	0.00	219.5820
2	0.00	6.8714
3	0.45	-383.0310
4	0.20	2.5164

[5 rows x 21 columns]

```
[2]: # Ver la información del dataset
df.describe(include="all")
```

```
[2]:
```

	Row ID	Order ID	Order Date	Ship Date	Ship Mode \
count	9994.000000	9994	9994	9994	9994

unique	NaN	5009	1237	1334	4
top	NaN	CA-2017-100111	9/5/2016	12/16/2015	Standard Class
freq	NaN	14	38	35	5968
mean	4997.500000	NaN	NaN	NaN	NaN
std	2885.163629	NaN	NaN	NaN	NaN
min	1.000000	NaN	NaN	NaN	NaN
25%	2499.250000	NaN	NaN	NaN	NaN
50%	4997.500000	NaN	NaN	NaN	NaN
75%	7495.750000	NaN	NaN	NaN	NaN
max	9994.000000	NaN	NaN	NaN	NaN

	Customer ID	Customer Name	Segment	Country	City \
count	9994	9994	9994	9994	9994
unique	793	793	3	1	531
top	WB-21850	William Brown	Consumer	United States	New York City
freq	37	37	5191	9994	915
mean	NaN	NaN	NaN	NaN	NaN
std	NaN	NaN	NaN	NaN	NaN
min	NaN	NaN	NaN	NaN	NaN
25%	NaN	NaN	NaN	NaN	NaN
50%	NaN	NaN	NaN	NaN	NaN
75%	NaN	NaN	NaN	NaN	NaN
max	NaN	NaN	NaN	NaN	NaN

	...	Postal Code	Region	Product ID	Category \
count	...	9994.000000	9994	9994	9994
unique	...	NaN	4	1862	3
top	...	NaN	West	OFF-PA-10001970	Office Supplies
freq	...	NaN	3203	19	6026
mean	...	55190.379428	NaN	NaN	NaN
std	...	32063.693350	NaN	NaN	NaN
min	...	1040.000000	NaN	NaN	NaN
25%	...	23223.000000	NaN	NaN	NaN
50%	...	56430.500000	NaN	NaN	NaN
75%	...	90008.000000	NaN	NaN	NaN
max	...	99301.000000	NaN	NaN	NaN

	Sub-Category	Product Name	Sales	Quantity	Discount \
count	9994	9994	9994.000000	9994.000000	9994.000000
unique	17	1850	NaN	NaN	NaN
top	Binders	Staple envelope	NaN	NaN	NaN
freq	1523	48	NaN	NaN	NaN
mean	NaN	NaN	229.858001	3.789574	0.156203
std	NaN	NaN	623.245101	2.225110	0.206452
min	NaN	NaN	0.444000	1.000000	0.000000
25%	NaN	NaN	17.280000	2.000000	0.000000
50%	NaN	NaN	54.490000	3.000000	0.200000

75%	NaN	NaN	209.940000	5.000000	0.200000
max	NaN	NaN	22638.480000	14.000000	0.800000

	Profit
count	9994.000000
unique	NaN
top	NaN
freq	NaN
mean	28.656896
std	234.260108
min	-6599.978000
25%	1.728750
50%	8.666500
75%	29.364000
max	8399.976000

[11 rows x 21 columns]

```
[3]: # Limpieza de nulos
df = df.dropna()
df
```

```
[3]:      Row ID      Order ID  Order Date  Ship Date  Ship Mode \
0         1  CA-2016-152156   11/8/2016  11/11/2016   Second Class
1         2  CA-2016-152156   11/8/2016  11/11/2016   Second Class
2         3  CA-2016-138688   6/12/2016   6/16/2016   Second Class
3         4  US-2015-108966  10/11/2015  10/18/2015   Standard Class
4         5  US-2015-108966  10/11/2015  10/18/2015   Standard Class
...
9989    9990  CA-2014-110422   1/21/2014   1/23/2014   Second Class
9990    9991  CA-2017-121258   2/26/2017   3/3/2017   Standard Class
9991    9992  CA-2017-121258   2/26/2017   3/3/2017   Standard Class
9992    9993  CA-2017-121258   2/26/2017   3/3/2017   Standard Class
9993    9994  CA-2017-119914   5/4/2017   5/9/2017   Second Class
```

	Customer ID	Customer Name	Segment	Country	City \
0	CG-12520	Claire Gute	Consumer	United States	Henderson
1	CG-12520	Claire Gute	Consumer	United States	Henderson
2	DV-13045	Darrin Van Huff	Corporate	United States	Los Angeles
3	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale
4	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale
...	...	...	...	...	...
9989	TB-21400	Tom Boeckenhauer	Consumer	United States	Miami
9990	DB-13060	Dave Brooks	Consumer	United States	Costa Mesa
9991	DB-13060	Dave Brooks	Consumer	United States	Costa Mesa
9992	DB-13060	Dave Brooks	Consumer	United States	Costa Mesa
9993	CC-12220	Chris Cortes	Consumer	United States	Westminster

	...	Postal Code	Region	Product ID	Category	Sub-Category	\
0	...	42420	South	FUR-BO-10001798	Furniture	Bookcases	
1	...	42420	South	FUR-CH-10000454	Furniture	Chairs	
2	...	90036	West	OFF-LA-10000240	Office Supplies	Labels	
3	...	33311	South	FUR-TA-10000577	Furniture	Tables	
4	...	33311	South	OFF-ST-10000760	Office Supplies	Storage	
...	...	...	...	...	...	...	
9989	...	33180	South	FUR-FU-10001889	Furniture	Furnishings	
9990	...	92627	West	FUR-FU-10000747	Furniture	Furnishings	
9991	...	92627	West	TEC-PH-10003645	Technology	Phones	
9992	...	92627	West	OFF-PA-10004041	Office Supplies	Paper	
9993	...	92683	West	OFF-AP-10002684	Office Supplies	Appliances	

		Product Name	Sales	Quantity	\
0		Bush Somerset Collection Bookcase	261.9600	2	
1	Hon Deluxe Fabric Upholstered Stacking Chairs,...		731.9400	3	
2	Self-Adhesive Address Labels for Typewriters b...		14.6200	2	
3	Bretford CR4500 Series Slim Rectangular Table		957.5775	5	
4	Eldon Fold 'N Roll Cart System		22.3680	2	
...	...	...	...	...	
9989		Ultra Door Pull Handle	25.2480	3	
9990	Tenex B1-RE Series Chair Mats for Low Pile Car...		91.9600	2	
9991		Aastra 57i VoIP phone	258.5760	2	
9992	It's Hot Message Books with Stickers, 2 3/4" x 5"		29.6000	4	
9993	Acco 7-Outlet Masterpiece Power Center, Wihtou...		243.1600	2	

	Discount	Profit
0	0.00	41.9136
1	0.00	219.5820
2	0.00	6.8714
3	0.45	-383.0310
4	0.20	2.5164
...	...	...
9989	0.20	4.1028
9990	0.00	15.6332
9991	0.20	19.3932
9992	0.00	13.3200
9993	0.00	72.9480

[9994 rows x 21 columns]

1.2 Paso 1: Preparar los Datos

Objetivo: Extraer el año de la columna de fechas y agregar las ventas y el beneficio por año.

Instrucciones: 1. Convierte la columna 'Order Date' a tipo fecha con `pd.to_datetime(df['Order Date'])`. 2. Crea una nueva columna 'Year' extrayendo el

año: `df['Year'] = df['Order Date'].dt.year`. 3. Agrupa el DataFrame por 'Year' y calcula:
- La suma de Sales. - La suma de Profit.

Usa `.agg({'Sales': 'sum', 'Profit': 'sum'})` y guarda en `df_agrupado`. Aplica `.reset_index()` al final. 4. Muestra `df_agrupado`. Deberías ver 4 filas (2014, 2015, 2016, 2017).

```
[4]: # Order Date a fecha y saco el año
# Uso sum en ventas y beneficio (los dos se acumulan, no se promedian)
df['Order Date'] = pd.to_datetime(df['Order Date'])
df['Year'] = df['Order Date'].dt.year

df_agrupado = df.groupby('Year').agg({
    'Sales': 'sum',
    'Profit': 'sum'
}).reset_index()
df_agrupado
```

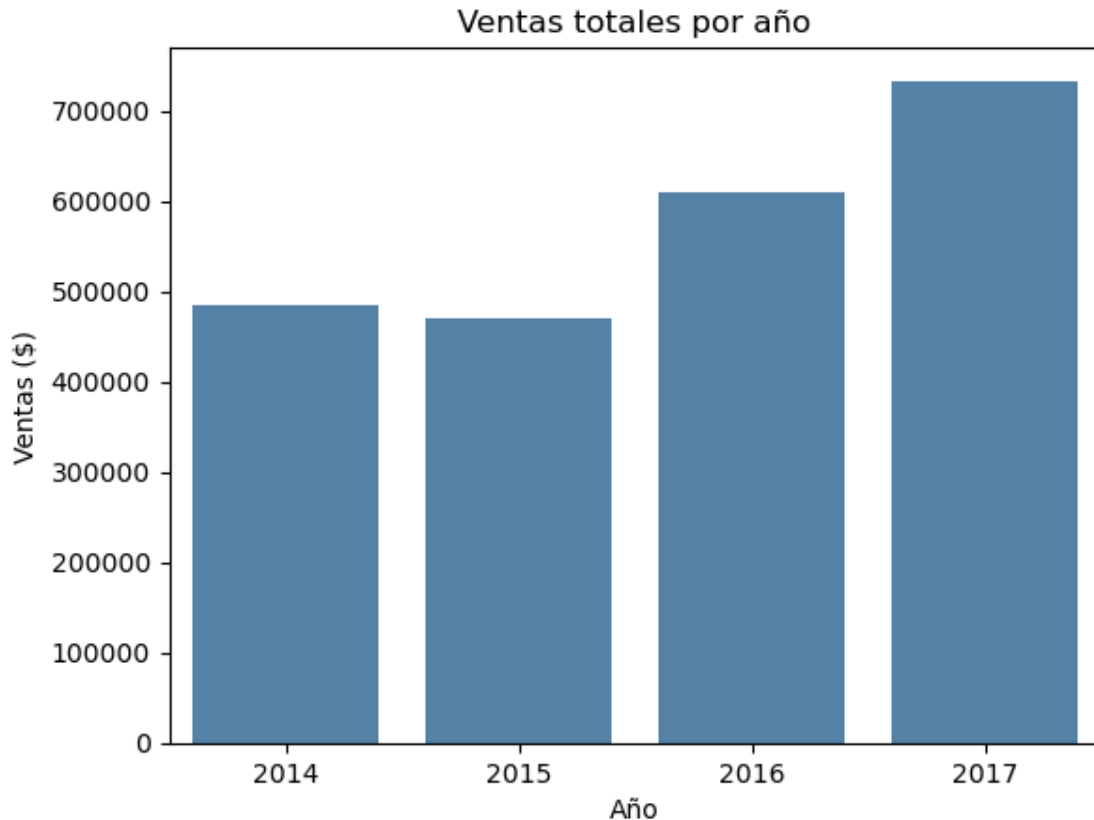
```
[4]:   Year      Sales      Profit
0  2014  484247.4981  49543.9741
1  2015  470532.5090  61618.6037
2  2016  609205.5980  81795.1743
3  2017  733215.2552  93439.2696
```

1.3 Paso 2: Gráfico de Barras Simple (Solo Ventas)

Objetivo: Visualizar únicamente las ventas por año con un gráfico de barras de Seaborn antes de añadir la línea de beneficio.

Instrucciones: 1. Usa `sns.barplot()` con `df_agrupado`. 2. Asigna `x='Year'` e `y='Sales'`. Pon un color neutro (por ejemplo `color='steelblue'`). 3. Añade título y etiquetas de ejes con `plt.title(...)`, `plt.xlabel(...)`, `plt.ylabel(...)`. 4. Muestra el gráfico. ¿Se aprecia la tendencia de crecimiento en ventas?

```
[5]: # grafico de barras
sns.barplot(
    x='Year',
    data=df_agrupado,
    y='Sales',
    color='steelblue'           # color uniforme: aquí no hay categoría que
    ↪diferenciar
)
plt.title('Ventas totales por año')
plt.xlabel('Año')
plt.ylabel('Ventas ($)')
plt.show()
```



1.4 Paso 3: Gráfico Combinado con Seaborn + Matplotlib (twinx)

Objetivo: Superponer la línea de beneficio sobre las barras de ventas usando un eje Y secundario, de modo que ambas métricas sean legibles a la vez.

Instrucciones: 1. Crea la figura con `fig, ax1 = plt.subplots(figsize=(8, 4))`. 2. Dibuja las barras en `ax1` con `sns.barplot(data=df_agrupado, x='Year', y='Sales', color='steelblue', alpha=0.8, ax=ax1)`. Añade `ax1.set_axisbelow(True)`. 3. Crea el segundo eje con `ax2 = ax1.twinx()`. 4. Dibuja la línea de beneficio en `ax2` con `sns.lineplot(data=df_agrupado, x='Year', y='Profit', ax=ax2, color='darkorange', marker='D', linewidth=2)`. 5. Etiqueta el eje derecho con `ax2.set_ylabel('Beneficio ($)', color='darkorange')` y aplica `ax2.tick_params(axis='y', labelcolor='darkorange')` para que los números del eje también sean naranjas. 6. Desactiva la cuadrícula del eje secundario con `ax2.grid(visible=False)` para que no se solape con la del primario. 7. Añade un título y muestra el gráfico.

```
[6]: # Combinado: barras de Sales + línea de Profit en eje Y secundario
fig, ax1 = plt.subplots(figsize=(8, 4))

sns.barplot(
    data=df_agrupado,
```

```

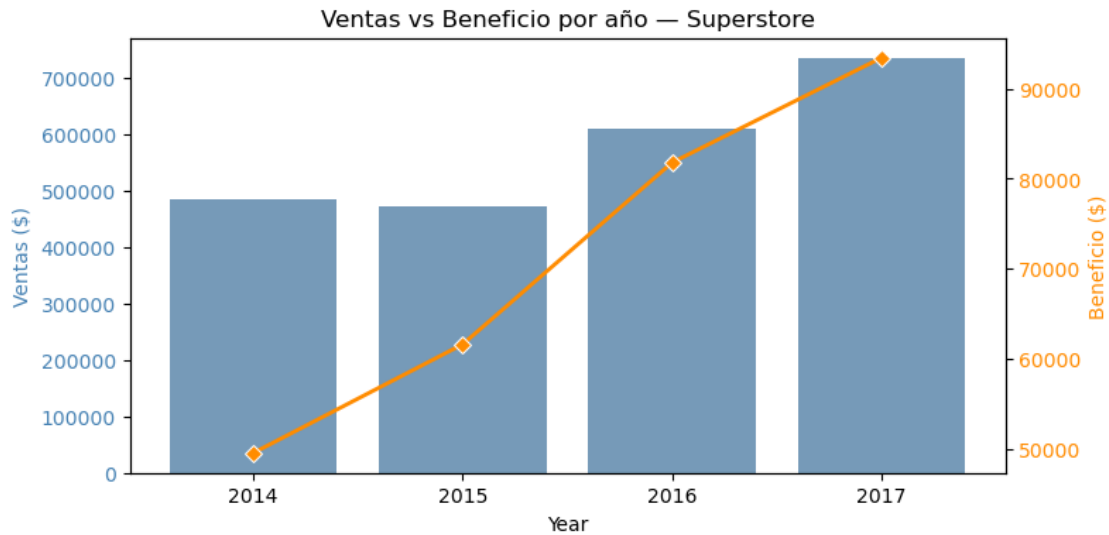
    x='Year',
    y='Sales',
    color='steelblue',
    alpha=0.8,          # transparencia para que se vea la línea encima
    ax=ax1
)
ax1.set_axisbelow(True)
ax1.set_ylabel('Ventas ($)', color='steelblue')
ax1.tick_params(axis='y', labelcolor='steelblue')

ax2 = ax1.twinx()

# Paso la Serie sola para que use el índice posicional 0..3 igual que las
↳ barras.
# Si paso x='Year' se va fuera del area de barras (barplot usa ticks categóricos
# y lineplot ticks numéricos).
line_color = 'darkorange'
ax2.set_ylabel('Beneficio ($)', color=line_color)
sns.lineplot(
    data=df_agrupado['Profit'],
    alpha=1,
    marker='D',          # diamante para resaltar cada punto
    ax=ax2,
    linewidth=2,
    color=line_color
)
ax2.tick_params(axis='y', labelcolor=line_color)
ax2.set_axisbelow(True)
ax2.grid(visible=False) # quito la rejilla del 2º eje para no duplicar
↳ la del primero

plt.title('Ventas vs Beneficio por año - Superstore')
plt.show()

```



1.5 Paso 4: Gráfico Combinado con Plotly por Categoría

Objetivo: Reproducir el gráfico con Plotly usando `make_subplots` con eje Y secundario, y añadir el desglose por categoría de producto en las barras para un análisis más detallado.

Instrucciones: 1. Crea un nuevo DataFrame `df_categoria` agrupando por `['Year', 'Category']` y calculando la suma de `Sales` y `Profit`. Aplica `.reset_index()`. 2. Crea también `df_profit_total` agrupando solo por `'Year'` y calculando la suma de `Profit`. Aplica `.reset_index()`. 3. Crea la figura con `fig = make_subplots(specs=[[{"secondary_y": True}]])`. 4. Con un bucle `for category in df_categoria['Category'].unique()`, filtra `df_categoria` por cada categoría y añade un `go.Bar(...)` a la figura con `secondary_y=False`. Activa `barmode='group'` en el layout. 5. Añade la línea de beneficio total con `go.Scatter(x=..., y=..., mode='markers+lines', name='Beneficio Total')` con `secondary_y=True`. 6. Configura el layout con títulos para ambos ejes Y y un título general. 7. Muestra el gráfico con `fig.show()`.

```
[7]: # Hacemos lo mismo pero por año y categoría (alimenta las barras agrupadas)
df_categoria = df.groupby(['Year', 'Category']).agg({
    'Sales': 'sum',
    'Profit': 'sum'
}).reset_index()

# Datos para la línea (Profit total por año, sin desglose)
df_profit_total = df.groupby('Year').agg({
    'Profit': 'sum'
}).reset_index()

# Crear un subplot con especificación para un segundo eje Y
fig = make_subplots(specs=[[{"secondary_y": True}]])
```



```

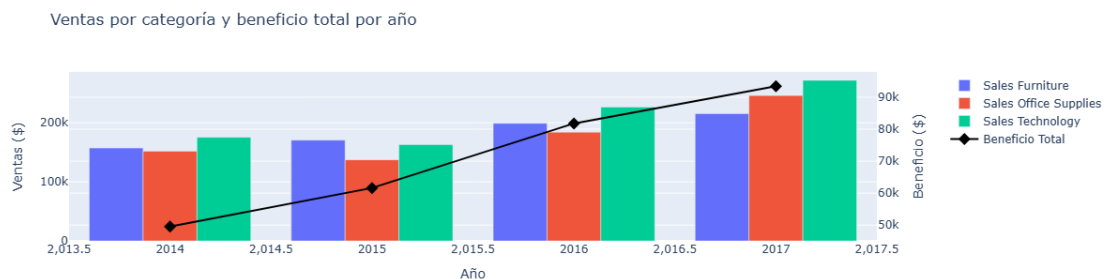
# Añadir el gráfico de barras para 'Sales' diferenciando por 'Category'
for category in df_categoria['Category'].unique():
    fig.add_trace(
        go.Bar(
            x=df_categoria[df_categoria['Category'] == category]['Year'],
            y=df_categoria[df_categoria['Category'] == category]['Sales'],
            name=f'Sales {category}',
            offsetgroup=category,
        ),
        secondary_y=False,
    )

# Añadir el gráfico de líneas para 'Profit' total
fig.add_trace(
    go.Scatter(
        x=df_profit_total['Year'],
        y=df_profit_total['Profit'],
        name='Beneficio Total',
        mode='markers+lines',
        line=dict(color='black', width=2), # negro para destacar sobre las
        ↪barras de colores
        marker=dict(symbol='diamond', size=10)
    ),
    secondary_y=True,
)

# Actualizar el layout del gráfico
fig.update_layout(
    title='Ventas por categoría y beneficio total por año',
    xaxis_title='Año',
    yaxis_title='Ventas ($)',
    yaxis2_title='Beneficio ($)',
    barmode='group', # barras agrupadas, no apiladas
)

fig.show()

```



1.5.1 Interpretación del Gráfico Combinado Barras y Líneas

Este gráfico final integra **dos niveles de análisis** en una sola figura:

1. **Las Barras (Ventas por Categoría):** permiten comparar, dentro de cada año, qué categoría de producto genera más ingresos. El eje izquierdo tiene la escala de ventas totales en dólares.
2. **La Línea (Beneficio Total):** muestra la evolución del beneficio neto en el eje derecho. Aunque las ventas crezcan, el beneficio puede no hacerlo proporcionalmente (por ejemplo, si aumentan los descuentos o los costes).
3. **Conclusión clave:** este gráfico es el estándar en cuadros de mando de negocio porque responde de un vistazo a la pregunta “¿Vendemos más y ganamos más?”, que no siempre tiene la misma respuesta.

1.6 Reflexión Final del Alumno

Tómate un rato para reflexionar. * El **gráfico combinado de barras y líneas** es uno de los más usados en entornos empresariales y de negocio porque permite comparar un volumen (barras) con una tasa o métrica relacionada (línea) en el mismo espacio visual. * Recuerda la regla de oro vista en clase: **no añadas más de dos líneas** al gráfico, o se volverá ilegible. Si necesitas mostrar más variables, considera dividirlo en varios subgráficos. * Como reto adicional: ¿podrías repetir el análisis agrupando por 'Region' en lugar de por 'Category'? ¿O cambiar el eje X a meses en lugar de años para ver la estacionalidad de las ventas?

Asegúrate de que cada celda tenga sus respectivos comentarios.

Por Región me quedan 4 barras, Central East South West, útil para pillar si alguna vende mucho pero pierde margen. Y por meses se vería la estacionalidad, pienso que noviembre y diciembre se disparan por navidad.

```
[8]: # Reto: mismo gráfico pero por Region en vez de Category
df_region = df.groupby(['Year', 'Region']).agg({
    'Sales': 'sum',
    'Profit': 'sum'
}).reset_index()

df_profit_total_r = df.groupby('Year').agg({
    'Profit': 'sum'
}).reset_index()

fig = make_subplots(specs=[[{"secondary_y": True}]])

# Una barra por cada región (Central, East, South, West)
for region in df_region['Region'].unique():
    fig.add_trace(
        go.Bar(
            x=df_region[df_region['Region'] == region]['Year'],
```

```

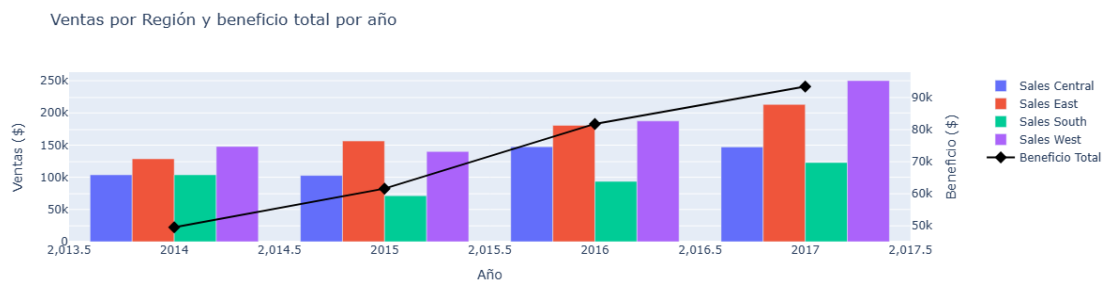
        y=df_region[df_region['Region'] == region]['Sales'],
        name=f'Sales {region}',
        offsetgroup=region,
    ),
    secondary_y=False,
)

fig.add_trace(
    go.Scatter(
        x=df_profit_total_r['Year'],
        y=df_profit_total_r['Profit'],
        name='Beneficio Total',
        mode='markers+lines',
        line=dict(color='black', width=2),
        marker=dict(symbol='diamond', size=10)
    ),
    secondary_y=True,
)

fig.update_layout(
    title='Ventas por Región y beneficio total por año',
    xaxis_title='Año',
    yaxis_title='Ventas ($)',
    yaxis2_title='Beneficio ($)',
    barmode='group',
)

fig.show()

```



09_tarea_heatmap_hexabin_diamonds

April 27, 2026

1 Tarea paso a paso: Heatmap 2D y Hexbin

¡ATENCIÓN! Lee esto antes de empezar: Esta tarea está diseñada paso a paso para que afiances los conceptos vistos en clase sobre los gráficos de densidad **Heatmap 2D** y **Hexbin**. Su objetivo es que practiques cómo visualizar la **densidad de datos** cuando tienes tantos puntos que un scatter plot clásico se convierte en una mancha ilegible.

Nota importante sobre la Inteligencia Artificial: Es tentador copiar y pegar el enunciado de estos ejercicios en ChatGPT o Claude para que te den la solución... **Hacer eso perjudicará gravemente tu aprendizaje. (CODEA UN POCO QUE TE ENTERES QUE HACES).** La única forma de asimilar la sintaxis de las herramientas de visualización es escribiéndolas. **Usa tu cerebro, actúa paso a paso, equivócate, lee la documentación de Seaborn y Pandas, busca tu fallo y ¡aprende de él!**

1.1 Preparación y Datos

Para esta tarea vamos a utilizar el dataset **“Diamonds”**, que recoge las características y precios de más de **53.000 diamantes**. Cada fila representa un diamante con su peso en quilates, su corte, color, claridad y precio en dólares.

Con más de 53.000 puntos, es el dataset ideal para ver el problema del **overplotting**: si intentamos hacer un scatter plot clásico de **carat vs price**, los puntos se solaparán tanto que será imposible saber dónde se concentran realmente los datos. Los gráficos de densidad (Heatmap 2D y Hexbin) son la solución.

1. Descarga el dataset de Kaggle: [Diamonds](#)
2. Guarda el archivo `diamonds.csv` en la carpeta `data/` de tu proyecto.
3. En la siguiente celda, importa las librerías necesarias y carga el CSV en un DataFrame llamado `df`.

```
[1]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# CARGA DE DATOS
df = pd.read_csv("data/diamonds.csv")

df.head(5)
```

```
[1]:   carat      cut color clarity depth table price      x      y      z
0    0.23    Ideal      E     SI2   61.5   55.0   326   3.95   3.98   2.43
1    0.21  Premium      E     SI1   59.8   61.0   326   3.89   3.84   2.31
2    0.23     Good      E     VS1   56.9   65.0   327   4.05   4.07   2.31
3    0.29  Premium      I     VS2   62.4   58.0   334   4.20   4.23   2.63
4    0.31     Good      J     SI2   63.3   58.0   335   4.34   4.35   2.75
```

```
[2]: # Ver la información del dataset
df.describe(include="all")
```

```
[2]:   count      carat      cut  color clarity      depth      table  \
count    53940.000000  53940  53940   53940  53940.000000  53940.000000
unique           NaN      5      7      8           NaN           NaN
top           NaN    Ideal      G     SI1           NaN           NaN
freq           NaN  21551  11292  13065           NaN           NaN
mean          0.797940     NaN     NaN     NaN        61.749405    57.457184
std           0.474011     NaN     NaN     NaN         1.432621     2.234491
min           0.200000     NaN     NaN     NaN        43.000000    43.000000
25%           0.400000     NaN     NaN     NaN        61.000000    56.000000
50%           0.700000     NaN     NaN     NaN        61.800000    57.000000
75%           1.040000     NaN     NaN     NaN        62.500000    59.000000
max           5.010000     NaN     NaN     NaN        79.000000    95.000000

count      price      x      y      z
count    53940.000000  53940.000000  53940.000000  53940.000000
unique           NaN           NaN           NaN           NaN
top           NaN           NaN           NaN           NaN
freq           NaN           NaN           NaN           NaN
mean       3932.799722     5.731157     5.734526     3.538734
std       3989.439738     1.121761     1.142135     0.705699
min        326.000000     0.000000     0.000000     0.000000
25%        950.000000     4.710000     4.720000     2.910000
50%       2401.000000     5.700000     5.710000     3.530000
75%       5324.250000     6.540000     6.540000     4.040000
max      18823.000000    10.740000    58.900000    31.800000
```

```
[3]: # Limpieza de nulos
df = df.dropna()
df
```

```
[3]:   carat      cut color clarity depth table price      x      y      z
0    0.23    Ideal      E     SI2   61.5   55.0   326   3.95   3.98   2.43
1    0.21  Premium      E     SI1   59.8   61.0   326   3.89   3.84   2.31
2    0.23     Good      E     VS1   56.9   65.0   327   4.05   4.07   2.31
3    0.29  Premium      I     VS2   62.4   58.0   334   4.20   4.23   2.63
4    0.31     Good      J     SI2   63.3   58.0   335   4.34   4.35   2.75
...    ...    ...    ...    ...    ...    ...    ...    ...    ...
```

53935	0.72	Ideal	D	SI1	60.8	57.0	2757	5.75	5.76	3.50
53936	0.72	Good	D	SI1	63.1	55.0	2757	5.69	5.75	3.61
53937	0.70	Very Good	D	SI1	62.8	60.0	2757	5.66	5.68	3.56
53938	0.86	Premium	H	SI2	61.0	58.0	2757	6.15	6.12	3.74
53939	0.75	Ideal	D	SI2	62.2	55.0	2757	5.83	5.87	3.64

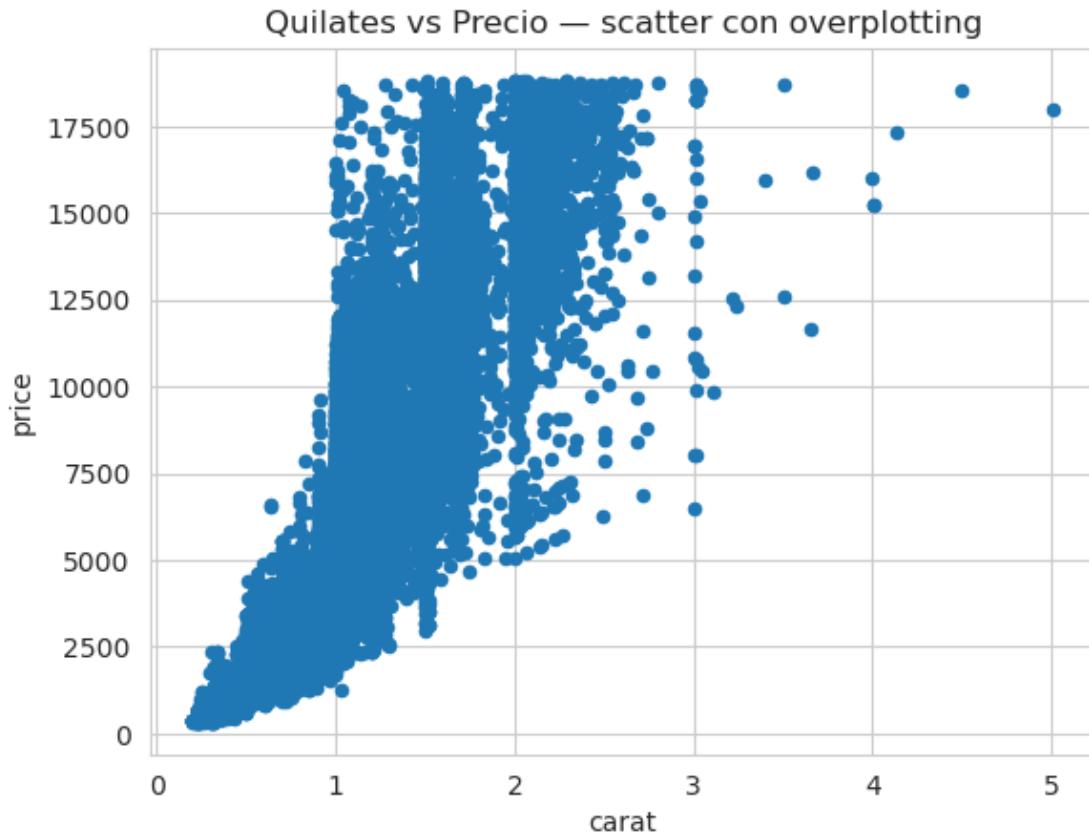
[53940 rows x 10 columns]

1.2 Paso 1: El Problema — Scatter Plot con Overplotting

Objetivo: Ver con tus propios ojos por qué el scatter plot clásico falla con datasets grandes y necesitamos una alternativa de densidad.

Instrucciones: 1. Usa `df.plot.scatter()` para hacer un scatter plot con `x='carat'` e `y='price'`. 2. Añade un título con `plt.title(...)` y muestra el gráfico. 3. Observa el resultado: ¿puedes saber dónde se concentran la mayoría de los diamantes? ¿O todo es una mancha azul oscura? Ese es el problema del **overplotting**.

```
[4]: # carat vs price con todos los puntos (53k) - para ver el problema del
      ↪overplotting
      # https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.plot.scatter.
      ↪html
      sns.set_style("whitegrid")
      df.plot.scatter(
          x='carat',
          y='price'
      )
      plt.title('Quilates vs Precio - scatter con overplotting')
      plt.show()
```



1.2.1 Interpretación

Con 53.000 puntos superpuestos, el scatter plot es casi inútil: no se distingue dónde están la mayoría de los diamantes. Necesitamos un gráfico que muestre la **densidad** de puntos en cada zona, no los puntos individuales.

1.3 Paso 2: Heatmap 2D con Jointplot (kind='hist')

Objetivo: Usar el `jointplot` de Seaborn con `kind='hist'` para dividir el espacio en celdas rectangulares y colorear cada celda según la densidad de puntos que contiene.

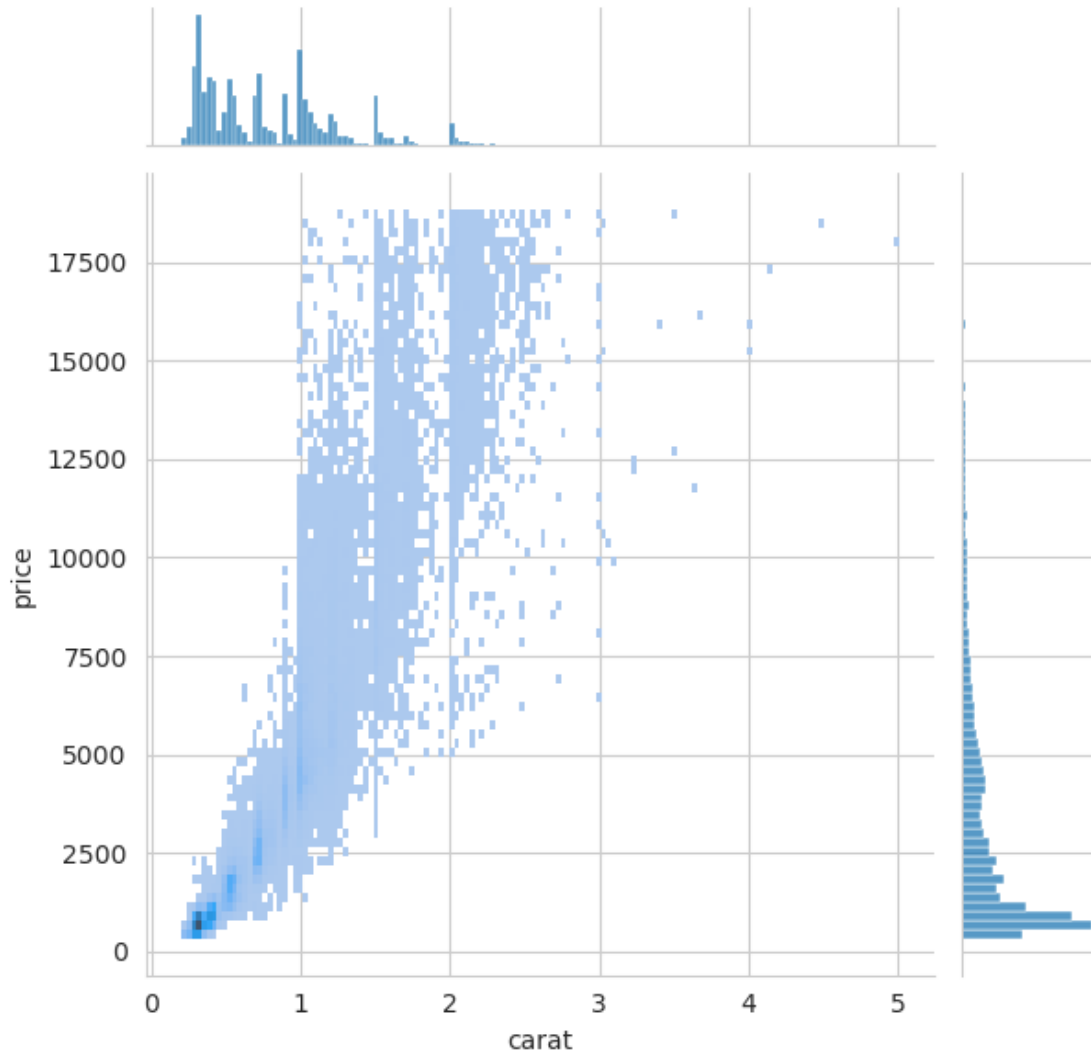
Instrucciones: 1. Usa `sns.jointplot()` con `data=df`, `x='carat'`, `y='price'` y `kind='hist'`. 2. Añade `plt.show()` al final. 3. Observa cómo las celdas más oscuras indican las zonas donde se concentran más diamantes. Los gráficos laterales muestran la distribución marginal de cada variable por separado.

```
[5]: # Heatmap de datos numéricos (rectángulos + histogramas marginales)
# https://seaborn.pydata.org/generated/seaborn.jointplot.html
sns.jointplot(
    data=df,
    x='carat',
```

```

    y='price',
    kind='hist'
)
plt.show()

```

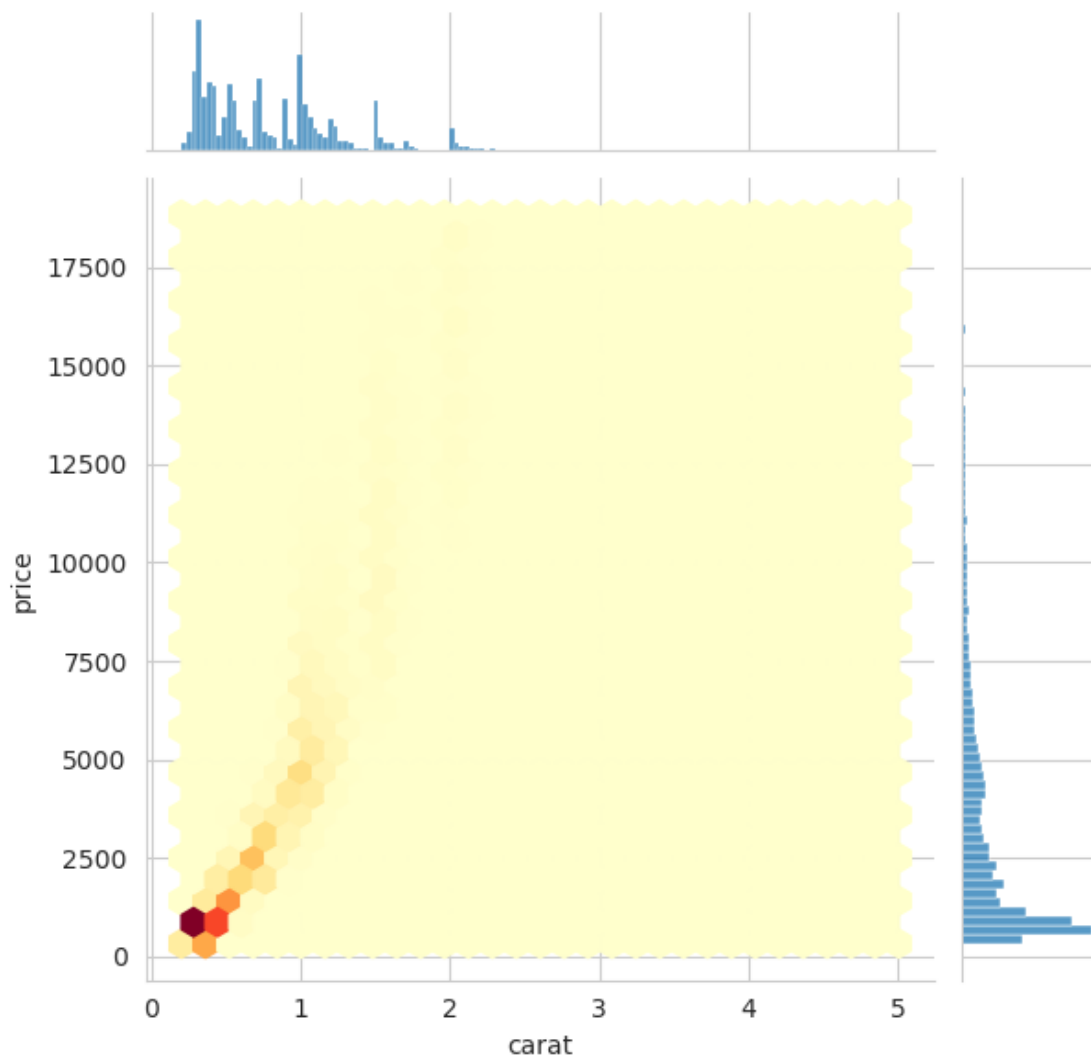


1.4 Paso 3: Hexbin con Jointplot (kind='hex')

Objetivo: Sustituir las celdas rectangulares por hexágonos, que tesslan el plano de forma más eficiente y son visualmente más claros para representar densidad.

Instrucciones: 1. Repite el `sns.jointplot()` del Paso 2 pero cambia `kind='hex'`. 2. Dentro del parámetro `joint_kws` (un diccionario), añade: - `'gridsize': 30` para controlar el tamaño de los hexágonos (más alto = hexágonos más pequeños y más detalle). - `'cmap': 'YlOrRd'` para usar una paleta de colores de amarillo a rojo (amarillo = poca densidad, rojo = mucha densidad). 3. Muestra el gráfico. ¿Qué rango de quilates y precios concentra más diamantes?


```
[6]: # Heatmap de datos numéricos en versión hexagonal
# https://seaborn.pydata.org/generated/seaborn.jointplot.html
sns.jointplot(
    data=df,
    x='carat',
    y='price',
    kind='hex',
    joint_kws={
        'gridsize': 30,          # más alto = hexágonos más pequeños y más
    detail
        'cmap': 'YlOrRd'        # paleta amarillo-rojo para densidad
    }
)
plt.show()
```

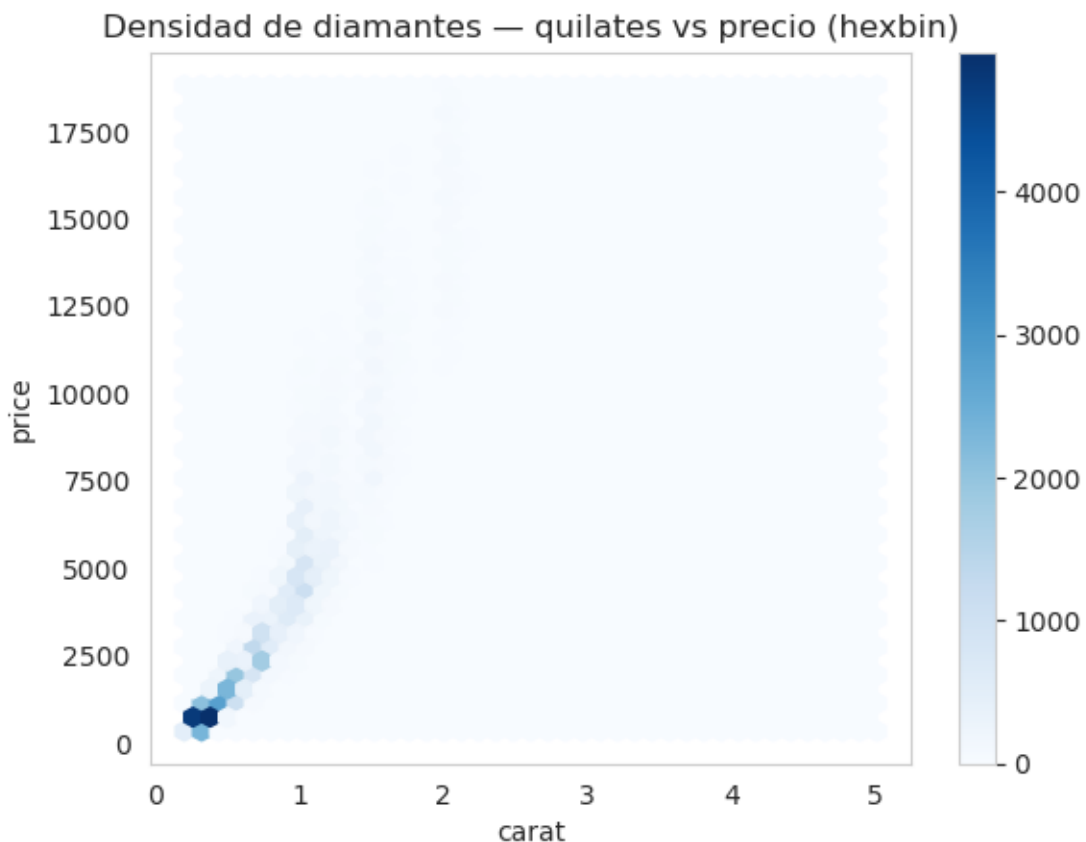


1.5 Paso 4: Hexbin con Pandas y Personalización

Objetivo: Crear el hexbin directamente desde Pandas con `df.plot.hexbin()`, que es la forma más rápida para exploración inicial, y personalizar el mapa de colores y el tamaño de la cuadrícula.

Instrucciones: 1. Usa `df.plot.hexbin()` con `x='carat'`, `y='price'` y `gridsize=40`. 2. Añade el parámetro `cmap='Blues'` para usar una paleta de azules (o prueba `'inferno'`, `'plasma'`, `'magma'`). 3. Desactiva la cuadrícula de fondo con `plt.grid(False)` para que no interfiera visualmente con los hexágonos. 4. Añade un título descriptivo y muestra el gráfico. 5. (Opcional): prueba a cambiar `gridsize` a 15 y a 60 y observa cómo afecta al nivel de detalle.

```
[7]: # Hexabin de datos numéricos
# https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.plot.hexbin.html
df.plot.hexbin(
    x='carat',
    y='price',
    gridsize=40,                                # algo más fino que el paso anterior
    cmap='Blues'
)
plt.grid(False)                                # sin rejilla, que compite con los hexágonos
plt.title('Densidad de diamantes - quilates vs precio (hexbin)')
plt.show()
```



1.5.1 Interpretación del Hexbin

Los gráficos de densidad revelan lo que el scatter plot ocultaba:

1. **La gran mayoría de los diamantes** se concentra en la zona de **0.3 a 1.0 quilates** y **500 a 5.000 dólares**. Esa es la zona de mayor demanda del mercado.
2. **A mayor número de quilates, mayor precio**, pero la relación no es lineal: los diamantes de más de 2 quilates son muy escasos (hexágonos aislados en la esquina superior derecha).
3. El **parámetro gridsize** controla la resolución: un valor bajo da una visión general (hexágonos grandes), un valor alto da más detalle pero puede ser ruidoso si los datos son escasos en alguna zona.

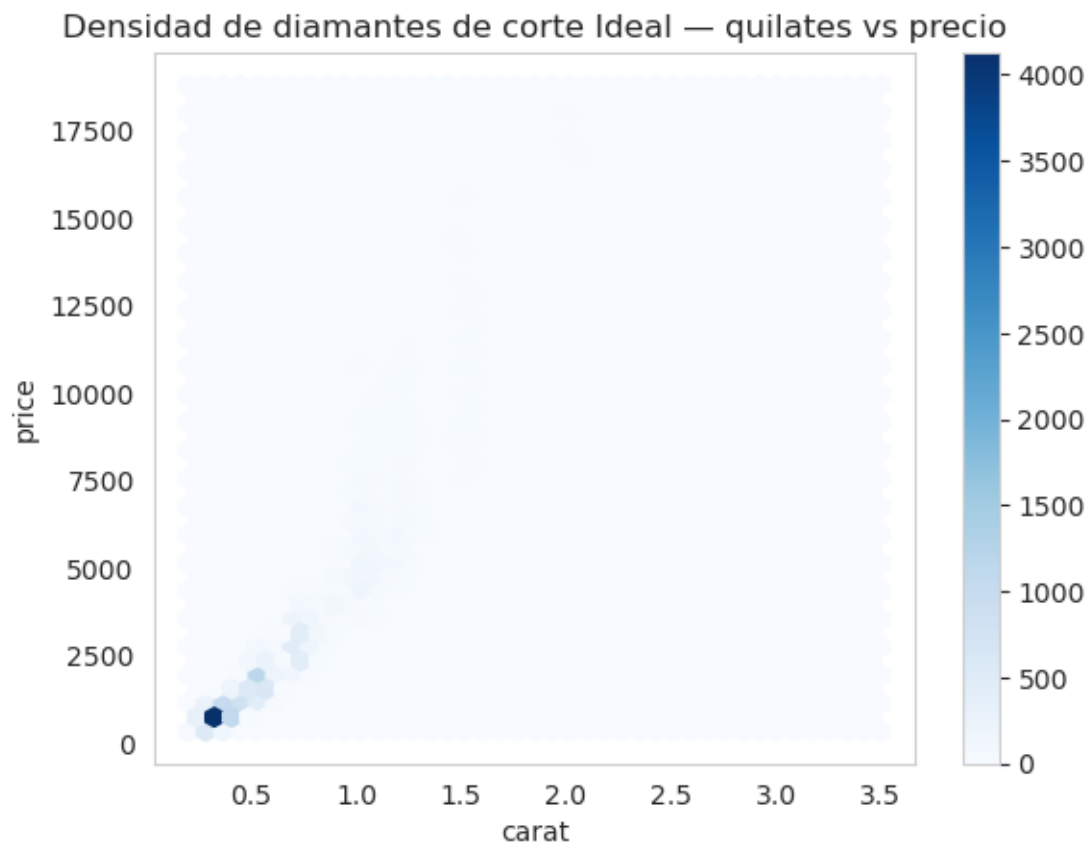
1.6 Reflexión Final del Alumno

Tómate un rato para reflexionar. * El **Heatmap 2D** y el **Hexbin** son los gráficos de referencia cuando trabajas con datasets con **más de 10.000 filas** y quieres estudiar la relación entre dos variables continuas. El scatter plot clásico colapsa por overplotting, pero estos gráficos revelan la estructura real de los datos. * La diferencia entre ambos es geométrica: el histograma 2D usa **rectángulos** (más fácil de leer en cuadrículas alineadas) y el hexbin usa **hexágonos** (tesslan el plano sin dejar huecos y tienen distancia uniforme al centro en todas las direcciones). * Como reto adicional: ¿podrías repetir el hexbin pero filtrando solo los diamantes con `cut == 'Ideal'`? ¿La distribución de quilates y precios cambia respecto al dataset completo?

Asegúrate de que cada celda tenga sus respectivos comentarios.

La forma de la nube no cambiaría mucho, pero se iría un poco arriba en Y. A igualdad de quilates un Ideal se paga más caro que un corte inferior.

```
[8]: # Reto: hexbin solo con cut == 'Ideal' (espero la nube algo más arriba en Y, ↵  
      ↪son más caros)  
df_ideal = df[df['cut'] == 'Ideal']  
df_ideal.plot.hexbin(  
    x='carat',  
    y='price',  
    gridsize=40,  
    cmap='Blues'  
)  
plt.grid(False)  
plt.title("Densidad de diamantes de corte Ideal - quilates vs precio")  
plt.show()
```



10_tarea_heatmap_categorico_videogames

April 27, 2026

1 Tarea paso a paso: Heatmap Categórico

¡ATENCIÓN! Lee esto antes de empezar: Esta tarea está diseñada paso a paso para que afiances los conceptos vistos en clase sobre los **Heatmaps Categóricos**. Su objetivo es que practiques cómo cruzar dos variables categóricas y representar su relación numérica mediante una cuadrícula de colores, detectando patrones que con un gráfico de barras serían imposibles de ver.

Nota importante sobre la Inteligencia Artificial: Es tentador copiar y pegar el enunciado de estos ejercicios en ChatGPT o Claude para que te den la solución... **Hacer eso perjudicará gravemente tu aprendizaje. (CODEA UN POCO QUE TE ENTERES QUE HACES).** La única forma de asimilar la sintaxis de las herramientas de visualización es escribiéndolas. **Usa tu cerebro, actúa paso a paso, equivócate, lee la documentación de Seaborn, busca tu fallo y ¡aprende de él!**

1.1 Preparación y Datos

Para esta tarea vamos a utilizar el dataset **“Video Game Sales”**, que recoge las ventas globales de más de 16.000 videojuegos desde 1980 hasta 2016. Cada fila representa un juego con su plataforma (PS4, Xbox, PC...), género (Action, Sports, RPG...) y ventas en millones de unidades en distintas regiones.

Es un dataset perfecto para un **Heatmap Categórico** porque cruzar **Género** con **Plataforma** genera una tabla de doble entrada que sería ilegible como gráfico de barras múltiples, pero que el heatmap convierte en un vistazo inmediato.

1. Descarga el dataset de Kaggle: [Video Game Sales](#)
2. Guarda el archivo `vgsales.csv` en la carpeta `data/` de tu proyecto.
3. En la siguiente celda, importa las librerías necesarias y carga el CSV en un DataFrame llamado `df`.

```
[1]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# CARGA DE DATOS
df = pd.read_csv("data/vgsales.csv")

df.head(5)
```

```
[1]:
```

	Rank	Name	Platform	Year	Genre	Publisher	\
0	1	Wii Sports	Wii	2006.0	Sports	Nintendo	
1	2	Super Mario Bros.	NES	1985.0	Platform	Nintendo	
2	3	Mario Kart Wii	Wii	2008.0	Racing	Nintendo	
3	4	Wii Sports Resort	Wii	2009.0	Sports	Nintendo	
4	5	Pokemon Red/Pokemon Blue	GB	1996.0	Role-Playing	Nintendo	

	NA_Sales	EU_Sales	JP_Sales	Other_Sales	Global_Sales
0	41.49	29.02	3.77	8.46	82.74
1	29.08	3.58	6.81	0.77	40.24
2	15.85	12.88	3.79	3.31	35.82
3	15.75	11.01	3.28	2.96	33.00
4	11.27	8.89	10.22	1.00	31.37

```
[2]: # Ver la información del dataset
df.describe(include="all")
```

```
[2]:
```

	Rank	Name	Platform	Year	\
count	16598.000000	16598	16598	16327.000000	
unique	NaN	11493	31	NaN	
top	NaN	Need for Speed: Most Wanted	DS	NaN	
freq	NaN	12	2163	NaN	
mean	8300.605254	NaN	NaN	2006.406443	
std	4791.853933	NaN	NaN	5.828981	
min	1.000000	NaN	NaN	1980.000000	
25%	4151.250000	NaN	NaN	2003.000000	
50%	8300.500000	NaN	NaN	2007.000000	
75%	12449.750000	NaN	NaN	2010.000000	
max	16600.000000	NaN	NaN	2020.000000	

	Genre	Publisher	NA_Sales	EU_Sales	JP_Sales	\
count	16598	16540	16598.000000	16598.000000	16598.000000	
unique	12	578	NaN	NaN	NaN	
top	Action	Electronic Arts	NaN	NaN	NaN	
freq	3316	1351	NaN	NaN	NaN	
mean	NaN	NaN	0.264667	0.146652	0.077782	
std	NaN	NaN	0.816683	0.505351	0.309291	
min	NaN	NaN	0.000000	0.000000	0.000000	
25%	NaN	NaN	0.000000	0.000000	0.000000	
50%	NaN	NaN	0.080000	0.020000	0.000000	
75%	NaN	NaN	0.240000	0.110000	0.040000	
max	NaN	NaN	41.490000	29.020000	10.220000	

	Other_Sales	Global_Sales
count	16598.000000	16598.000000
unique	NaN	NaN
top	NaN	NaN

freq	NaN	NaN
mean	0.048063	0.537441
std	0.188588	1.555028
min	0.000000	0.010000
25%	0.000000	0.060000
50%	0.010000	0.170000
75%	0.040000	0.470000
max	10.570000	82.740000

```
[3]: # Limpieza de nulos
df = df.dropna()
df
```

```
[3]:
```

	Rank	Name	Platform	\
0	1	Wii Sports	Wii	
1	2	Super Mario Bros.	NES	
2	3	Mario Kart Wii	Wii	
3	4	Wii Sports Resort	Wii	
4	5	Pokemon Red/Pokemon Blue	GB	
...	...	...	...	
16593	16596	Woody Woodpecker in Crazy Castle 5	GBA	
16594	16597	Men in Black II: Alien Escape	GC	
16595	16598	SCORE International Baja 1000: The Official Game	PS2	
16596	16599	Know How 2	DS	
16597	16600	Spirits & Spells	GBA	

	Year	Genre	Publisher	NA_Sales	EU_Sales	JP_Sales	\
0	2006.0	Sports	Nintendo	41.49	29.02	3.77	
1	1985.0	Platform	Nintendo	29.08	3.58	6.81	
2	2008.0	Racing	Nintendo	15.85	12.88	3.79	
3	2009.0	Sports	Nintendo	15.75	11.01	3.28	
4	1996.0	Role-Playing	Nintendo	11.27	8.89	10.22	
...	...	...	...	...	...	...	
16593	2002.0	Platform	Kemco	0.01	0.00	0.00	
16594	2003.0	Shooter	Infogrames	0.01	0.00	0.00	
16595	2008.0	Racing	Activision	0.00	0.00	0.00	
16596	2010.0	Puzzle	7G//AMES	0.00	0.01	0.00	
16597	2003.0	Platform	Wanadoo	0.01	0.00	0.00	

	Other_Sales	Global_Sales
0	8.46	82.74
1	0.77	40.24
2	3.31	35.82
3	2.96	33.00
4	1.00	31.37
...	...	...
16593	0.00	0.01

16594	0.00	0.01
16595	0.00	0.01
16596	0.00	0.01
16597	0.00	0.01

[16291 rows x 11 columns]

1.2 Paso 1: Preparar la Tabla Pivote (Género × Plataforma)

Objetivo: Transformar el DataFrame en una tabla de doble entrada (pivote) que cruce **Genre** y **Platform** sumando las ventas globales. Esta tabla es la que alimentará el heatmap.

Instrucciones: 1. Para no generar un heatmap demasiado grande, filtra el DataFrame para quedarte solo con las **10 plataformas con más ventas totales**. Para ello: - Calcula las ventas totales por plataforma: `df.groupby('Platform')['Global_Sales'].sum()`. - Obtén los nombres de las 10 plataformas top con `.nlargest(10).index`. - Filtra el DataFrame: `df_filtrado = df[df['Platform'].isin(top_plataformas)]`. 2. Agrupa `df_filtrado` por `['Genre', 'Platform']` y calcula la **suma** de `Global_Sales`. Guarda en `agrupado`. 3. Crea la tabla pivote con `agrupado.unstack()`. Si es necesario, aplica `.droplevel(0, axis=1)` para quitar el nivel extra de columnas. 4. Muestra la tabla pivote resultante.

```
[4]: # Top 10 plataformas por ventas globales (filtro para que el heatmap no quede
      ↪ilegible)
top_plataformas = df.groupby('Platform')['Global_Sales'].sum().nlargest(10).
      ↪index
df_filtrado = df[df['Platform'].isin(top_plataformas)]

# Agrupamos datos por género y plataforma
agrupado = df_filtrado.groupby(['Genre', 'Platform']).agg({
    'Global_Sales': 'sum'          # sum porque me interesa el volumen total,
    ↪no la media
})

pivote = agrupado.unstack().droplevel(0, axis=1)
pivote
```

```
[4]: Platform      DS      GBA      PC      PS      PS2      PS3      PS4      PSP \
Genre
Action      114.16  54.26  30.67  125.74  272.43  304.02  87.06  62.66
Adventure    47.15  12.10  10.09   20.77   21.16   22.87   4.70  10.69
Fighting      7.20   4.21   0.14   72.68   89.19   51.70   8.04  21.88
Misc      137.67  28.50   8.41   44.90   98.69   45.91   7.40  13.70
Platform      77.40  78.08   0.49   64.21   72.11   29.85   7.01  17.28
Puzzle       83.87  12.09   0.92   12.08    5.90    0.46   0.02   5.52
Racing       38.58  18.80   3.80  102.89  154.21   73.10  11.53  34.73
Role-Playing 126.56  64.21  47.37   78.30   91.55   75.24  25.77  49.01
Shooter       8.20   3.60  43.44   39.31  108.28  195.80  75.32  19.77
```


Simulation	131.65	5.91	51.73	25.33	42.26	10.72	0.77	6.28
Sports	31.71	16.41	12.01	119.51	262.64	134.91	50.07	39.90
Strategy	14.76	7.45	45.63	21.67	15.04	4.77	0.41	10.29

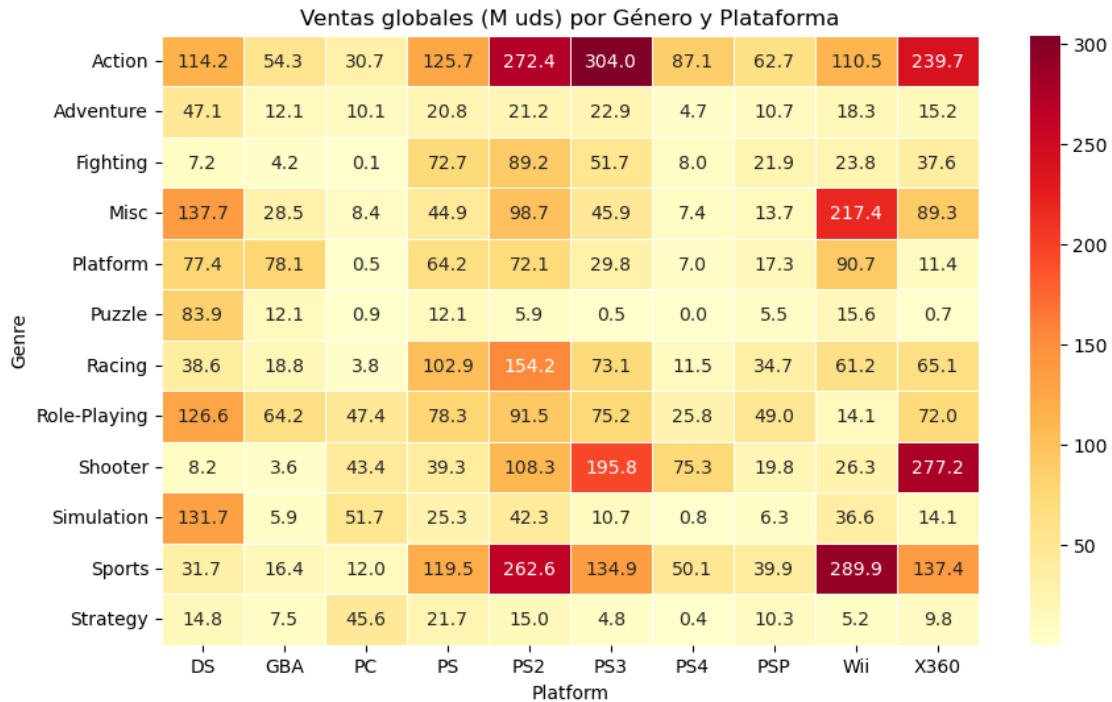
Platform	Wii	X360
Genre		
Action	110.48	239.67
Adventure	18.33	15.23
Fighting	23.82	37.64
Misc	217.43	89.33
Platform	90.68	11.39
Puzzle	15.63	0.71
Racing	61.24	65.13
Role-Playing	14.06	71.97
Shooter	26.34	277.23
Simulation	36.62	14.10
Sports	289.95	137.43
Strategy	5.23	9.77

1.3 Paso 2: Heatmap Básico con Anotaciones

Objetivo: Crear el heatmap más sencillo posible a partir de la tabla pivote, añadiendo los valores numéricos dentro de cada celda para facilitar la lectura.

Instrucciones: 1. Usa `sns.heatmap()` pasándole la tabla pivote del Paso 1. 2. Añade `annot=True` para que aparezca el valor numérico dentro de cada celda. 3. Añade `fmt='.1f'` para que los números se muestren con un decimal. 4. Añade `linewidth=0.5` para separar visualmente las celdas. 5. Muestra el gráfico. ¿Qué género y plataforma combinan las mayores ventas globales?

```
[5]: # Heatmap de variables categóricas que codifican una variable numérica
# https://seaborn.pydata.org/generated/seaborn.heatmap.html
plt.figure(figsize=(10, 6))
sns.heatmap(
    pivote,
    annot=True,
    cmap='YlOrRd',
    linewidth=0.5,
    fmt='.1f'
)
plt.title('Ventas globales (M uds) por Género y Plataforma')
plt.show()
```



1.4 Paso 3: Heatmap Temporal (Género × Década)

Objetivo: Construir un heatmap temporal que muestre cómo han evolucionado las ventas de cada género a lo largo del tiempo, agrupando los años por décadas.

Instrucciones:

1. Convierte la columna Year a entero: `df['Year'] = df['Year'].astype(int)`.
2. Crea una nueva columna Decada calculando la década de cada año: `df['Decada'] = (df['Year'] // 10) * 10`. Esto convierte 1995 → 1990, 2003 → 2000, etc.
3. Agrupa por ['Genre', 'Decada'] y calcula la suma de Global_Sales. Aplica `.unstack()` y `.droplevel(0, axis=1)` para obtener la tabla pivote. Guarda en `pivote_temporal`.
4. Crea el heatmap con `plt.figure(figsize=(10, 6))` y `sns.heatmap(pivote_temporal, cmap='YlOrRd', linewidth=0.3, annot=True, fmt='.0f')`.
5. Muestra el gráfico. ¿En qué décadas vendieron más los juegos de Action? ¿Y los de Sports?

[6]: *# Year a int y saco la década (1995 -> 1990, 2003 -> 2000, etc.)*

```
df['Year'] = df['Year'].astype(int)
df['Decada'] = (df['Year'] // 10) * 10

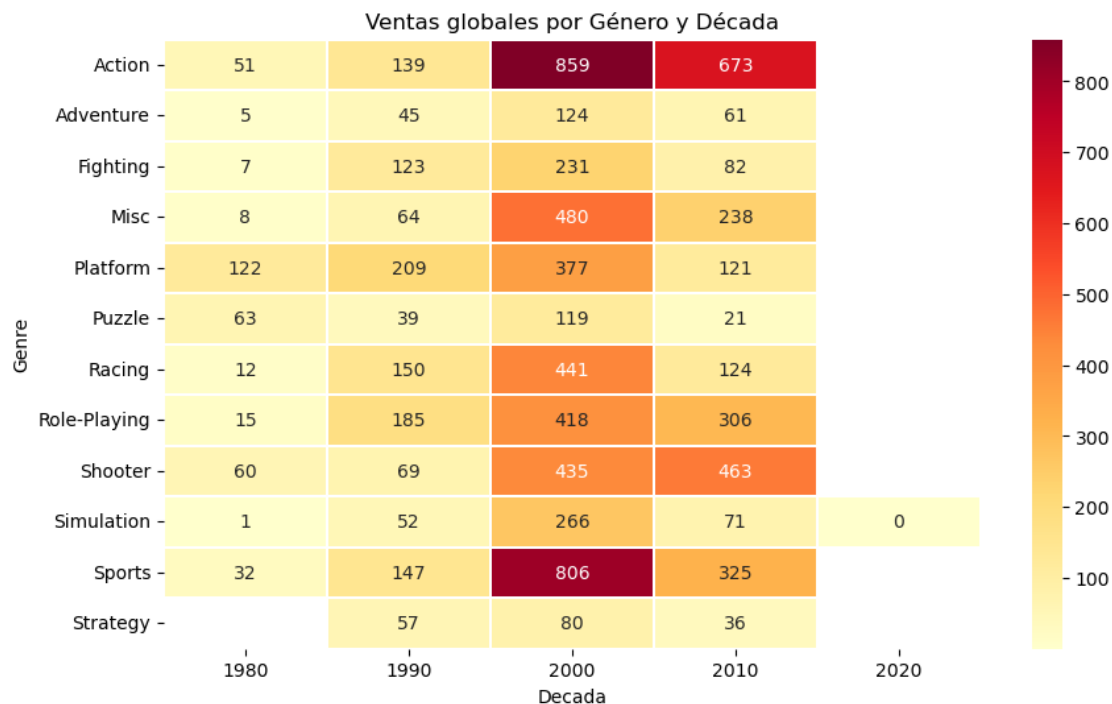
# Agrupamos datos por género y década
pivote_temporal = df.groupby(['Genre', 'Decada']).agg({
    'Global_Sales': 'sum'
}).unstack().droplevel(0, axis=1)

# Heatmap temporal
plt.figure(figsize=(10, 6))
```

```

sns.heatmap(
    pivote_temporal,
    cmap='YlOrRd',
    linewidth=0.3,
    annot=True,
    fmt='.0f'
    # sin decimales, las cifras por década son
    ↪ grandes
)
plt.title('Ventas globales por Género y Década')
plt.show()

```



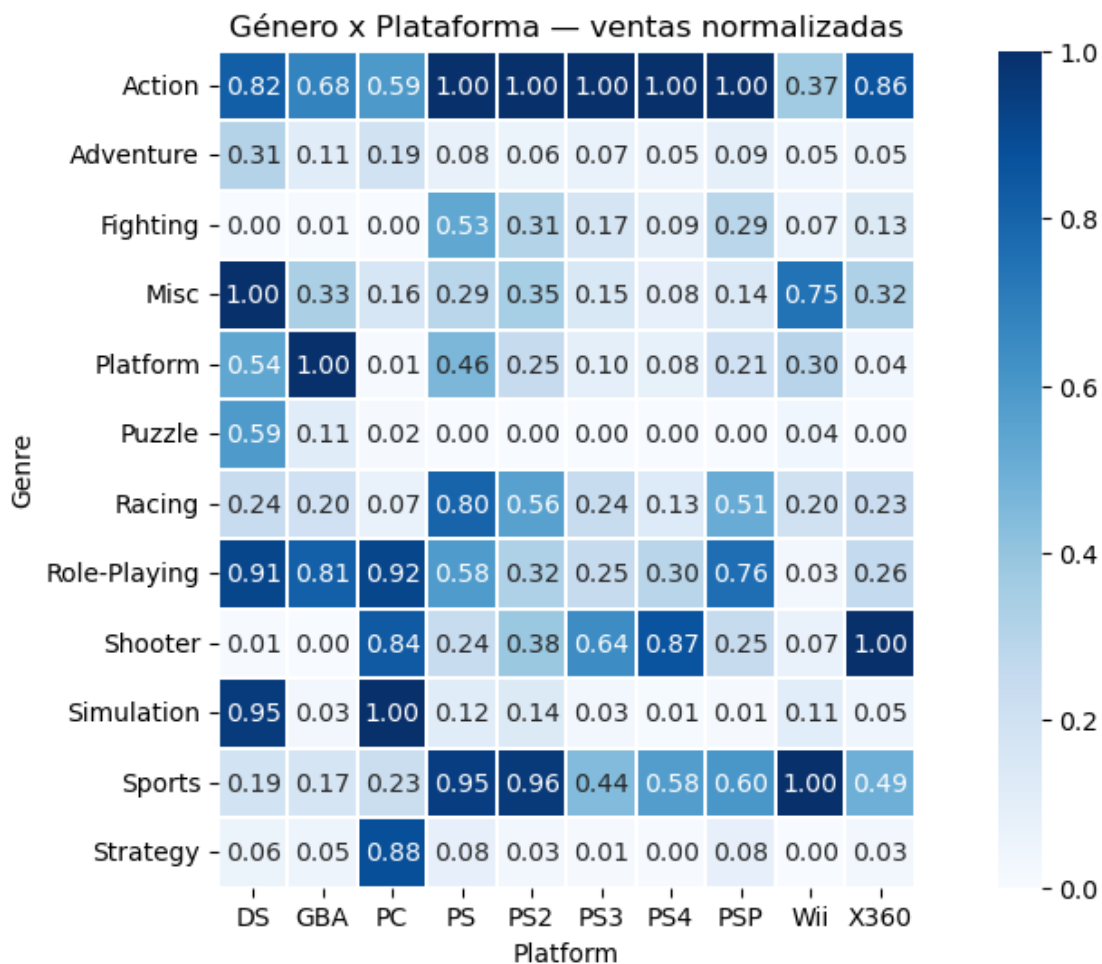
1.5 Paso 4: Heatmap Normalizado y Comparación con Barras

Objetivo: Normalizar la tabla pivote del Paso 1 para comparar géneros en igualdad de condiciones (independientemente de su volumen total de ventas), y comparar visualmente el heatmap con un gráfico de barras múltiples para la misma información.

Instrucciones: 1. Normaliza la tabla pivote del Paso 1 con Min-Max por columna (plataforma): $\text{pivote_norm} = (\text{pivote} - \text{pivote.min()}) / (\text{pivote.max()} - \text{pivote.min()})$. Rellena los NaN con 0 usando `.fillna(0)`. 2. Haz un heatmap estilizado con `plt.figure(figsize=(12, 6))`, `cmap='Blues'`, `square=True`, `linewidth=0.2` y `annot=True`, `fmt='.2f'`. Ponle un título descriptivo. 3. En una nueva celda, intenta representar la **misma información** (Genre × Platform × ventas) con `sns.barplot()` usando `hue='Platform'` y `x='Genre'`. Rota las etiquetas del eje X con `ax.tick_params(axis='x', labelrotation=45)`. 4. Compara ambos gráficos: ¿cuál transmite mejor los patrones de un vistazo cuando hay muchas categorías?

```
[7]: # Min-Max por columna (cada plataforma se compara consigo misma)
# Así PS2/X360 no aplastan visualmente al resto
pivote_norm = (pivote - pivote.min()) / (pivote.max() - pivote.min())
pivote_norm = pivote_norm.fillna(0)

# Heatmap de dos variables categóricas que codifican una variable numérica
plt.figure(figsize=(12, 6))
ax = sns.heatmap(
    pivote_norm,
    square=True, # Me genera cuadraditos simetricos
    cmap='Blues',
    linewidth=0.2,
    annot=True,
    fmt='.2f' # 2 decimales porque ahora todo está en [0,1]
)
plt.title('Género x Plataforma - ventas normalizadas')
plt.show()
```

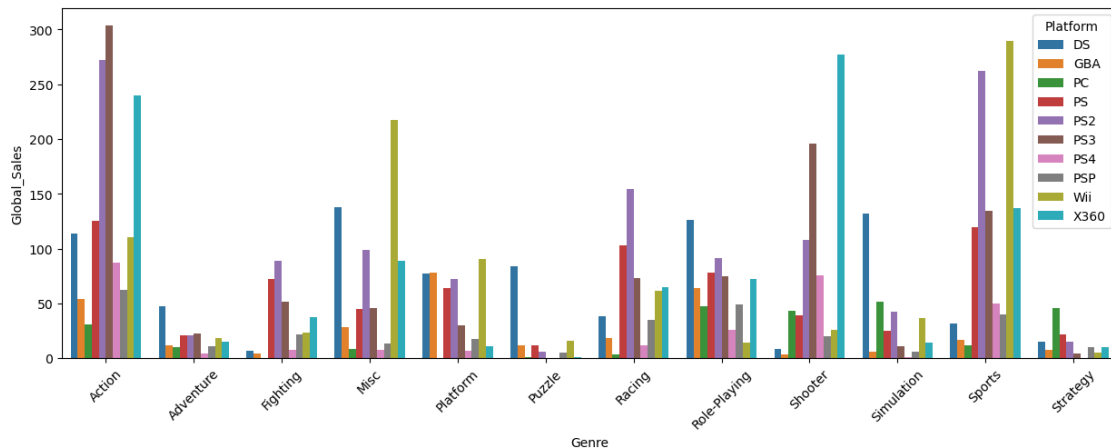


```
[8]: # Intentando hacer un grafico de barras multiples con la misma informacion
plt.figure(figsize=(15, 5))
temp_df = agrupado.reset_index()

ax = sns.barplot(
    data=temp_df,
    x='Genre',
    y='Global_Sales',
    hue='Platform',
    linewidth=0
)

# Rotacion de las etiquetas del eje X
ax.tick_params(axis='x', labelrotation=45)

plt.show()
```



1.5.1 Interpretación: Heatmap vs Gráfico de Barras

Ambos gráficos muestran **exactamente la misma información**, pero con resultados muy distintos:

- **El gráfico de barras múltiples** se vuelve muy difícil de leer cuando hay muchas categorías: las barras se amontonan, la leyenda se llena de colores y es complicado comparar la misma plataforma entre distintos géneros.
- **El heatmap** convierte esa misma tabla en un vistazo: el ojo humano detecta inmediatamente las celdas más oscuras (mayor venta) y los patrones por fila o columna, sin necesidad de leer valores uno a uno.
- **Regla general:** cuando tienes más de 4-5 categorías en cada eje, el heatmap gana al gráfico de barras múltiples.

1.6 Reflexión Final del Alumno

Tómate un rato para reflexionar. * El **Heatmap Categórico** es la herramienta perfecta para explorar tablas de contingencia: cualquier cruce de dos variables categóricas con un valor numérico asociado es un candidato natural para este gráfico. * Recuerda dos reglas importantes: (1) si las variables tienen **escalas muy distintas** entre sí, normaliza los datos antes de hacer el heatmap o una variable dominará todo el color; (2) incluye siempre `annot=True` cuando el número de celdas sea manejable, ya que los valores exactos complementan el color. * Como reto adicional: ¿podrías crear un heatmap cruzando **Genre** con **Publisher** (filtrando solo los 10 publishers con más ventas) para ver qué editorial domina en cada género?

Asegúrate de que cada celda tenga sus respectivos comentarios.

Cambio Platform por Publisher y ya. Apuesto a que Nintendo manda en Platform y Sports, y EA sube en Sports y Action.

```
[9]: # Reto: Género x top 10 Publishers (mismo esquema del paso 1 cambiando Platform_
    ↪ por Publisher)
top_publishers = df.groupby('Publisher')['Global_Sales'].sum().nlargest(10).
    ↪ index
df_pub = df[df['Publisher'].isin(top_publishers)]

# Agrupamos datos por género y publisher
agrupado_pub = df_pub.groupby(['Genre', 'Publisher']).agg({
    'Global_Sales': 'sum'
})

pivote_pub = agrupado_pub.unstack().droplevel(0, axis=1)

# Heatmap de variables categóricas que codifican una variable numérica
plt.figure(figsize=(12, 6))
sns.heatmap(
    pivote_pub,
    annot=True,
    fmt='.0f',
    linewidth=0.5,
    cmap='YlOrRd'
)
plt.title('Ventas globales por Género y top 10 Publishers')
plt.show()
```

